Dr. Shyama Prasad Mukherjee

International Institute of Information Technology, Naya Raipur

(A Joint Initiative of NTPC Ltd. & Government of Chhattisgarh)

Atal Nagar – Naya Raipur, Chhattisgarh – 493661

SYLLABUS BOOK

Bachelor of Technology (B.Tech.)

Computer Science and Engineering (CSE)

Curriculum & Detailed Syllabi

Batch 2025 Onwards • Total Credits: 160

(With effect from July 2025)

Designed under the Outcome-Based Education (OBE) framework
aligned with NBA & NEP 2020

Department of Computer Science & Engineering

Table of Contents

Table of Contents 2
1. About IIIT-NR 4
2. About the Department of CSE 5
3. Outcome-Based Education (OBE) Framework 7
3.1 Programme Educational Objectives (PEOs) and Justification 7
3.2 Programme Outcomes (POs) 7
3.3 Programme Specific Outcomes (PSOs) 8
3.4 PEO – PO / PSO Mapping 9
4. Industry Relevance of the Subjects 10
5. Detailed Scheme of Syllabi (Semesters 1–7) 11
6. Detailed Course-wise Syllabus (Core Subjects) 15
MA-L-01-C Linear Algebra and Matrix Analysis 16
MA-L-02-C Calculus 18
CS-L-01-C Problem Solving with C Programming 20
EC-L-01-C Digital Electronics using Verilog 22
EC-L-02-C Introduction to Internet of Things 24
PH-L-01-C Quantum Mechanics 26
MA-L-03-C Probability and Statistics 28
CS-L-02-C Data Structures 30
EC-L-03-C Signals and Systems 32
DA-L-02-C Introduction to AI & ML 34
MA-L-04-C Discrete Mathematics 36
CS-L-03-C Design and Analysis of Algorithms 38
CS-L-04-C Database Management Systems 40
CS-L-05-C Operating Systems 42
CS-L-06-C Object Oriented Programming 44
MA-L-05-C Optimization Techniques 46
EC-L-05-C Digital Communication Systems 48
MA-L-06-C Graph Theory 50
CS-L-07-C Computer Networks 52
DA-L-03-C Computer Graphics and Visualization 54
CS-L-08-C Embedded Systems Design 56
CS-L-09-C Applied Machine Learning 58
CS-L-10-C Finite Automata and Formal Languages 60
CS-L-11-C Computer Organization and Architecture 62
CS-L-13-C Cloud Computing 64
CS-L-14-C Cryptography 66

CS-L-15-C Deep Learning 68
CS-L-16-C Computer Vision ..... **Error! Bookmark not defined.**
CS-L-17-C Contemporary Practices in Software Engineering 72
CS-L-19-C Distributed Systems 74
CS-L-20-C Compiler Design 76
7. Professional Electives (Programme Electives) 78
8. Open Electives (Suggested) 80
9. Introduction to Finance (Common Course) 801
10. NEP in Brief 80
11. **Frameworks for Evaluation of Community Engagement Activities (NSS/ISR) / Extra and Co-Curricular Activities** 807

1. About IIIT-NR

Dr. Shyama Prasad Mukherjee International Institute of Information Technology, Naya Raipur (IIIT-NR) is a premier state-funded institute established in 2015 under the International Institute of Information Technology University Act, 2013 of the Government of Chhattisgarh. The institute is a joint initiative of the Government of Chhattisgarh and National Thermal Power Corporation (NTPC) Limited, a Maharatna Central Public Sector Undertaking under the Ministry of Power, Government of India.

Incorporated as a state university and recognized by the University Grants Commission (UGC) and approved by the All India Council for Technical Education (AICTE), IIIT-NR is committed to pursuing excellence in higher education, research, and development in Information Technology (IT) and associated disciplines. The institute firmly believes in knowledge dissemination and in imparting entrepreneurial skills to its students, enabling them to address real-world problems with confidence and creativity.

The institute is housed on a fully residential, fifty-acre green campus in Naya Raipur (Atal Nagar) — India's first planned green and smart city — approximately 23 km from Raipur and well connected by air, rail, and road. The serene environment blends natural beauty with state-of-the-art infrastructure, creating an ideal space for innovation and learning.

IIIT-NR offers undergraduate (B.Tech.), postgraduate (M.Tech.), and doctoral (Ph.D.) programmes in Computer Science and Engineering (CSE), Electronics and Communication Engineering (ECE), and Data Science and Artificial Intelligence (DSAI). These credit-based programmes are designed to foster core competence together with innovation and entrepreneurship, with a dynamic curriculum structure that keeps pace with the changing needs of industry and academia.

Vision of the Institute

To be a globally recognized institution of excellence in information technology and allied disciplines, nurturing innovators, entrepreneurs, and socially responsible technologists who contribute to national and global development.

Mission of the Institute

- To impart high-quality, value-based, and outcome-oriented technical education.
- To foster state-of-the-art, interdisciplinary research and product development.
- To cultivate entrepreneurial mind-sets and industry-ready professional skills.
- To engage with society and industry for sustainable and inclusive growth.

Key Highlights

Particular	Details
Established	2015 (Act 15 of 2013, Government of Chhattisgarh)
Type	State University (Joint venture of Govt. of Chhattisgarh & NTPC Ltd.)
Recognition	UGC-recognized; AICTE-approved
Campus	50-acre fully residential green campus, Naya Raipur (Atal Nagar)
Programmes	B.Tech. (CSE / ECE / DSAI), M.Tech., Ph.D.
Motto	Knowledge gives us Intense Peace

2. About the Department of CSE

The Department of Computer Science and Engineering (CSE) at IIIT-NR offers a four-year Bachelor of Technology programme that builds a strong foundation in the theory and practice of computing — spanning programming, data structures and algorithms, systems, databases, networks, and the rapidly growing areas of artificial intelligence, machine learning, and data science. The programme combines rigorous fundamentals with extensive hands-on laboratory work, design projects, and industry-aligned skills so that graduates are equipped to design, build, and reason about complex software and computing systems.

The department is taught by highly qualified and research-active faculty whose expertise spans artificial intelligence and machine learning, data science, computer networks and security, distributed and cloud systems, theoretical computer science, computer vision, and software engineering. Every semester integrates project-based learning, supported by well-equipped computing laboratories, high-performance computing resources, and the active mentorship of research scholars. AI-assisted full-stack prototyping is embedded from the first year, reflecting the institute's emphasis on innovation and entrepreneurship.

Vision of the Department

To emerge as a centre of excellence in computer science and engineering education and research, producing globally competent engineers, innovators, and entrepreneurs capable of solving complex computing problems for the benefit of society.

Mission of the Department

1. To provide a rigorous, outcome-based foundation in computer science and engineering with strong hands-on, design, and problem-solving exposure.
2. To promote research, innovation, and entrepreneurship in emerging areas such as artificial intelligence, data science, cyber-security, and distributed and intelligent systems.
3. To inculcate professional ethics, lifelong learning, and social responsibility among students.
4. To build strong linkages with industry, research organizations, and the wider community for mutually beneficial collaboration.

Programme Offered

B.Tech. in Computer Science and Engineering — 4 years / 8 semesters, 160 credits, with multiple entry-exit options, multidisciplinary minors, an Honors track, and a strong project and internship component, in line with NEP 2020 and the NBA outcome-based education framework.

Credit Structure and Break-up

The programme follows the NEP-based B.Tech. Common Format credit scheme, with a total programme credit requirement of 160 credits distributed across Lecture (L), Tutorial (T), Practical (P), and Scholarly Engagement Hours (S) components as defined below:

L — Lecture	1 Credit = 1 hour / week
T — Tutorial	1 Credit = 1 hour / week
P — Practical	1 Credit = 2 hours / week
S — Scholarly Engagement Hours	0 Credit = 2 hours / week

Salient Features of the Curriculum

- Choice-based credit system with Programme Electives (Data Science & AI, Networking & Security, and Embedded & Systems tracks), Open Electives, and Multidisciplinary Minors.
- AI-assisted full-stack prototyping, IoT, 3D printing, and hardware-prototyping skills embedded from the first year.
- Strong project spine: Mini Project, Project-I & II (with research methodology), Minor & Major Projects, and a mandatory summer internship.
- Honors option (additional 15 credits) for high-performing students with a research publication / patent requirement.
- Value-added courses on Indian Knowledge Systems, ethics, constitution, finance, and community service for holistic development.

3. Outcome-Based Education (OBE) Framework

The B.Tech. (CSE) curriculum is designed and delivered following the Outcome-Based Education (OBE) philosophy advocated by the National Board of Accreditation (NBA) and consistent with the Washington Accord. Under OBE, the curriculum, teaching-learning processes, and assessment are aligned to clearly defined outcomes at three levels — Programme Educational Objectives (PEOs), Programme Outcomes (POs) with Programme Specific Outcomes (PSOs), and Course Outcomes (COs).

The OBE Hierarchy

Outcome Level	Description & Time-frame
PEOs	Broad statements describing the career and professional accomplishments graduates are expected to attain within 3–5 years of graduation.
POs / PSOs	Statements of knowledge, skills, and attitudes students should possess at the time of graduation; PSOs are programme-specific competencies (CSE).
COs	Specific, measurable abilities a student acquires on completing an individual course; COs are mapped to POs/PSOs and assessed each semester.

Attainment is measured through direct assessment (continuous internal evaluation, end-semester examinations, laboratory and project rubrics) and indirect assessment (course-exit, programme-exit, alumni, and employer surveys). The closed feedback loop drives continuous quality improvement (CQI).

3.1 Programme Educational Objectives (PEOs) and Justification

Within a few years of graduation, B.Tech. (CSE) graduates of IIIT-NR will:

PEO	Statement	Justification
PEO-1	Establish themselves as competent professionals in software, IT, data-science, and allied industries, or pursue higher studies and research in reputed institutions.	Reflects the strong computing core (programming, DSA, systems, AI/ML) and project spine that build employability and research readiness.
PEO-2	Apply computing knowledge and modern tools to analyse, design, and implement sustainable and scalable software solutions to real-world and societal problems.	Supported by design-oriented courses, open-source tools, software-engineering practice, and SDG/environment components in the curriculum.
PEO-3	Demonstrate effective communication, teamwork, leadership, ethical conduct, and entrepreneurial thinking in multidisciplinary environments.	Addressed by HSMC courses (language, discourse, ethics, finance), entrepreneurship & design thinking, and team-based projects.
PEO-4	Engage in lifelong learning and adapt to emerging technologies such as AI/ML, cloud, cyber-security, distributed systems, and quantum computing.	Enabled by foundation in AI/ML and systems, emerging-technology electives, MOOC/NPTEL pathways, and research methodology training.

3.2 Programme Outcomes (POs)

On successful completion of the programme, graduates will be able to demonstrate the twelve NBA Graduate Attributes (PO-1 to PO-12):

PO	Attribute	Statement
PO-1	**Engineering knowledge**	Apply knowledge of mathematics, science, engineering fundamentals, and a computing specialization to the solution of complex engineering problems.
PO-2	**Problem analysis**	Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO-3	**Design/development of solutions**	Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.
PO-4	**Investigation of complex problems**	Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.
PO-5	**Modern tool usage**	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
PO-6	**The engineer and society**	Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal, and cultural issues and the consequent responsibilities relevant to professional engineering practice.
PO-7	**Environment and sustainability**	Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate knowledge of and need for sustainable development.
PO-8	**Ethics**	Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
PO-9	**Individual and team work**	Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.
PO-10	**Communication**	Communicate effectively on complex engineering activities with the engineering community and with society at large.
PO-11	**Project management and finance**	Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO-12	**Life-long learning**	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

3.3 Programme Specific Outcomes (PSOs)

In addition to the POs, CSE graduates will be able to:

PSO	Statement
PSO-1	Apply principles of programming, data structures, algorithms, and software engineering to design, develop, test, and maintain efficient and reliable software systems.
PSO-2	Design and manage computing systems — including operating systems, databases, computer networks, and distributed and cloud platforms — using modern tools and best practices.

PSO	Statement
PSO-3	Build intelligent, data-driven, and secure applications by applying artificial intelligence, machine learning, data science, and cyber-security techniques to real-world problems.

3.4 PEO – PO / PSO Mapping

The following matrix maps each PEO to the POs and PSOs it primarily contributes to (✓ = strong correlation).

PEO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
PEO-1	✓	✓	✓	✓	✓						✓	✓	✓	✓	✓
PEO-2	✓	✓	✓	✓	✓	✓	✓				✓		✓	✓	✓
PEO-3			✓			✓	✓	✓	✓	✓	✓				✓
PEO-4	✓	✓		✓	✓		✓					✓	✓	✓	✓

Course-level CO–PO/PSO mapping matrices are maintained by the department for every course and are reviewed each semester as part of the continuous quality improvement (CQI) process.

4. Industry Relevance of the Subjects

The CSE curriculum is intentionally aligned with the competencies sought by the software, IT-services, product, data-science, cloud, and cyber-security industries, as well as with national missions such as Digital India, the IndiaAI Mission, and Atmanirbhar Bharat. The table below maps subject clusters to their industry relevance.

Subject Cluster	Industry Relevance & Typical Roles
Mathematics & Theory core (Linear Algebra, Calculus, Probability & Statistics, Discrete Mathematics, Graph Theory, Optimization, Finite Automata)	Foundation for algorithm design, cryptography, machine learning, and the formal reasoning that underpins compilers, verification, and large-scale software systems.
Programming & Data Structures (Problem Solving with C, Data Structures, Object Oriented Programming, Design & Analysis of Algorithms)	Core to every software-engineering role; directly assessed in technical interviews and essential for building efficient, maintainable applications at product and service companies.
Systems (Operating Systems, Computer Organization & Architecture, Computer Networks, Distributed Systems, Compiler Design, Cloud Computing)	Backbone of system-software, backend, cloud, and infrastructure engineering at firms such as Google, Microsoft, Amazon, and Indian product startups.
Data & Intelligence (Introduction to AIML, Applied Machine Learning, Deep Learning, Computer Graphics & Visualization)	Powers AI/ML, data-science, computer-vision, and analytics roles across virtually every sector — from fintech and healthcare to e-commerce and research labs.
Data Management (Database Management Systems, Big-Data & Analytics electives)	Essential for data-engineering, backend, and analytics roles; SQL/NoSQL and data-modelling skills are in universal demand.
Security & Cryptography (Cryptography, Network & Cyber-Security electives)	Critical for cyber-security, secure-software, and blockchain roles, aligned with national priorities in digital trust and data protection.
Embedded & Hardware (Digital Electronics with Verilog, Embedded Systems Design, IoT, Robotics)	Enables embedded-software, IoT, and robotics roles in automotive, industrial-automation, and hardware-software co-design domains.
Professional skills (Entrepreneurship & Design Thinking, IT Workshop, Software Engineering, Finance, Communication, Ethics, IKS)	Build the workplace-readiness, innovation, and managerial competence valued by recruiters and essential for entrepreneurs and team leaders.

Course-level industry relevance is further detailed within each course syllabus in Section 6.

5. Detailed Scheme of Syllabi (Semesters 1–7)

Credit & contact-hour scheme of the B.Tech. (CSE) programme, Batch 2025 onwards. Total programme credits: 160.

Legend — L: Lecture (1 credit = 1 hr/week); T: Tutorial (1 credit = 1 hr/week); P: Practical (1 credit = 2 hr/week); S: Scholarly Engagement Hours (0 credit = 2 hr/week). Type — BSC: Basic Science; ESC: Engineering Science; PCC: Programme Core; PSC: Programme Specific Core; VSEC: Vocational/Skill Enhancement; HSMC: Humanities, Social Science & Mgmt; MDM: Multidisciplinary Minor; PEC/PE: Programme Elective; OE/OEC: Open Elective; ELC/EL: Experiential Learning; CCA/CEA: Co-/Extra-curricular.

Semester 1 — Total Credits: 21

Sl.	Course Name	Code	Type	Cr.	L-T-P-S
1	Linear Algebra and Matrix Analysis	MA-L-01-C	BSC	3	2-1-0-0
2	Calculus	MA-L-02-C	BSC	3	2-1-0-0
3	Problem Solving with C Programming	CS-L-01-C	ESC/VSEC	3	2-0-2-2
4	Digital Electronics using Verilog	EC-L-01-C	ESC	3	2-0-2-0
5	International Language Competency	HS-L-01-C	HSMC (AEC)	2	1-0-2-2
6	IT Workshop: Full Stack Prototyping with AI Support	DA-L-01-C	VSEC	2	1-0-2-2
7	Entrepreneurship and Design Thinking	HS-L-02-C	HSMC	2	1-0-2-2
8	Internet of Things	EC-L-02-C	VSEC	2	1-0-2-2
9	Environmental Studies	HS-L-03-C	HSMC (VEC)	1	1-0-0-0
	Total			**21**	**14-2-10**

Semester 2 — Total Credits: 21

Sl.	Course Name	Code	Type	Cr.	L-T-P-S
1	Quantum Mechanics	PH-L-01-C	BSC	3	2-1-0-0
2	Probability and Statistics	MA-L-03-C	BSC	3	2-1-0-0
3	Data Structures	CS-L-02-C	ESC/VSEC	3	2-0-2-2
4	Signals and Systems	EC-L-03-C	ESC	3	2-0-2-0
5	Introduction to AIML	DA-L-02-C	ESC/VSEC	2	1-0-2-2
6	Communication and Discourse Strategies	HS-L-04-C	HSMC (AEC)	2	1-0-2-2
7	Human Values and Professional Ethics	HS-L-05-C	HSMC (VEC)	2	2-0-0-2
8	3D Printing and Prototype Design	EC-L-04-C	VSEC	2	1-0-2-2
9	Constitution of India	HS-L-06-C	HSMC (VEC)	1	1-0-0-0
	Total			**21**	**13-2-12**

Semester 3 — Total Credits: 22

Sl.	Course Name	Code	Type	Cr.	L-T-P-S
1	Discrete Mathematics	MA-L-04-C	BSC	3	2-1-0-0
2	Design and Analysis of Algorithms	CS-L-03-C	PCC	3	2-0-2-2
3	Database Management Systems	CS-L-04-C	PCC	3	2-0-2-2
4	Operating Systems	CS-L-05-C	PCC	3	2-0-2-2
5	Object Oriented Programming	CS-L-06-C	PCC	2	1-0-2-2
6	Optimization Techniques	MA-L-05-C	PSC	2	2-0-0-0
7	Digital Communication Systems	EC-L-05-C	PSC	2	1-0-2-0
8	Multi-Disciplinary Minor-1	MD-L-xx-E	MDM	3	3-0-0-0
9	Introduction to Finance	HS-L-07-C	HSMC (AEC)	1	2-0-0-0
	Total			**22**	**17-1-10**

Semester 4 — Total Credits: 25

Sl.	Course Name	Code	Type	Cr.	L-T-P-S
1	Graph Theory	MA-L-06-C	PCC	2	2-0-0-0
2	Computer Networks	CS-L-07-C	PCC	3	2-0-2-2
3	Computer Graphics and Visualization	DA-L-03-C	PCC	3	2-0-2-2
4	Embedded Systems Design	CS-L-08-C	PCC	3	2-0-2-0
5	Applied Machine Learning	CS-L-09-C	PCC	3	2-0-2-0
6	Finite Automata and Formal Languages	CS-L-10-C	PCC	3	2-2-0-2
7	Computer Organization and Architecture	CS-L-11-C	PCC	2	2-0-0-0
8	Multi-Disciplinary Minor-2	MD-L-xx-E	MDM	3	3-0-0-0
9	Experiential Learning through Mini Project	CS-L-12-C	EL	2	0-0-2-2
10	Community Services (NSS / ISR Activities)	HS-L-08-C	HSMC (CEA)	1	0-0-0-2
	Total			**25**	**15-0-12**

Semester 5 — Total Credits: 22

Sl.	Course Name	Code	Type	Cr.	L-T-P-S
1	Cloud Computing	CS-L-13-C	PCC	3	2-0-2-2
2	Cryptography	CS-L-14-C	PCC	3	2-0-2-2
3	Deep Learning	CS-L-15-C	PCC	3	2-0-2-0
4	Computer Vision	CS-L-16-C	PCC	3	2-0-2-2
5	Contemporary Practices in Software Engineering	CS-L-17-C	PCC	3	2-0-2-2
6	Project - I (Research Methodology)	CS-L-18-C	ELC	2	0-0-4-2
7	Multi-Disciplinary Minor-3	MD-L-xx-E	MDM	3	3-0-0-0
8	Course on Indian Knowledge Systems	HS-L-09-C	HSMC (AEC)	2	1-0-2-0
	Total			**22**	**13-0-16**

Semester 6 — Total Credits: 25

Sl.	Course Name	Code	Type	Cr.	L-T-P-S
1	Distributed Systems	CS-L-19-C	PCC	3	2-0-2-2
2	Compiler Design	CS-L-20-C	PCC	2	1-0-2-0
3	Project - II	CS-L-21-C	ELC	2	0-0-4-0
4	Open Elective - I	OE-L-xx-E	OE	3	3-0-0-0
5	Program Elective - I	PE-L-xx-E	PEC	3	3-0-0-0
6	Program Elective - II	PE-L-xx-E	PEC	3	3-0-0-0
7	Program Elective - III	PE-L-xx-E	PEC	3	3-0-0-0
8	Multi-Disciplinary Minor-4	MD-L-xx-E	MDM	3	3-0-0-0
9	Indian/Foreign Language	HS-L-10-C	HSMC (AEC)	1	1-0-2-0
10	Co-Curricular & Extracurricular Activities	HS-L-11-C	CCA	2	-
	Total			**25**	**15-0-8**

Summer Semester — Internship (ELC, 3 credits): evaluated at the beginning of Semester 7; credit and grades reflected in the 7th-semester grade sheet.

Semester 7 — Total Credits: 19

Sl.	Course Name	Code	Type	Cr.	L-T-P-S
1	Minor Project	CS-L-22-C	ELC	2	0-0-4-0
2	Program Elective - IV	PE-L-xx-E	PEC	3	3-0-0-0
3	Program Elective - V	PE-L-xx-E	PEC	3	3-0-0-0
4	Open Elective - II	OE-L-xx-E	OEC	3	3-0-0-0
5	Multi-Disciplinary Minor-5	MD-L-xx-E	MDM	3	3-0-0-0
6	IKS Elective (History of Indian Civilizations / Kautilya's Arthashastra / Vedic Mathematics / Vedic Corpus / Panini's Grammar)	HS-L-12-C	HSMC (IKS)	1	2-0-0-0
7	Internship (Summer Semester)	CS-L-23-C	ELC	3	0-0-0-6
	Total			**18**	**14-0-4**

Semester 8 — Total Credits: 6

Sl.	Course Name	Code	Type	Cr.	L-T-P-S
1	Major Project (may be completed as Internship-cum-Project; courses may be joined via MOOC/NPTEL)	CS-L-24-C	ELC	6	-
	Total			**6**	**-**

Exit Options & Honors

- Exit after Year 1, Year 2, or Year 3 is permitted as per ordinance, subject to additional skill-based courses (6 credits) completed in summer, as per the list floated by the department. Year-1 exit requires skill courses related to Web Development and Hardware Prototyping.
- Honors: an additional credential earned through 15 extra credits in the discipline (minimum 8 CGPI, no backlog); students may opt for it after the 3rd semester. The 8th-semester project must be academic, with at least one Tier I/II conference or SCI-journal paper communicated, or one patent filed.

6. Detailed Course-wise Syllabus (Core Subjects)

This section presents the detailed Outcome-Based Education (OBE) syllabus for the core (compulsory) courses of the programme, covering the Basic Science (BSC), Engineering Science (ESC), Programme Core (PCC), and Programme Specific Core (PSC) courses from Semester 1 to Semester 6. Each course is described using a uniform OBE template comprising: course information, Course Outcomes (COs), a five-module detailed syllabus with contact hours, text and reference books, industry relevance, Open Educational Resources (OERs) and open repositories, suggested laboratory exercises, and recommended open-source simulators/tools.

Professional (Programme) Electives and Open Electives are listed separately in Sections 7 and 8. Value-added, HSMC, and skill courses follow institute-wide common syllabi and are reflected in the scheme. Multidisciplinary Minor (MDM) streams are excluded as per the scope of this book.

MA-L-01-C Linear Algebra and Matrix Analysis

Course Code	MA-L-01-C	**Credits**	3
Category	BSC (Basic Science)	**Structure (L-T-P-S)**	2-1-0-0
Prerequisite	Nil	**Laboratory**	No

Provides the matrix-algebraic and vector-space foundation underpinning signal processing, communications, control, machine learning, and numerical computation in electronics engineering.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Solve systems of linear equations using Gaussian elimination and matrix inversion.
CO2	Carry out matrix operations including inverses, rank, and determinants.
CO3	Determine eigenvalues and eigenvectors and solve eigenvalue problems.
CO4	Apply matrix algebra to linear transformations and change of basis.
CO5	Demonstrate understanding of inner-product spaces, orthonormalization, and SVD.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Systems of Linear Equations** System of linear equations; Gaussian elimination; inverse and rank of a matrix by elementary transformations; properties and applications of determinants.	5
M2	**Vector Spaces** Vector spaces over real and complex fields; subspaces; linear dependence and independence; spanning set; basis and dimension.	6
M3	**Linear Transformations** Linear transformations and their matrices; kernel and range; change of basis; rank-nullity theorem; matrix representation.	6
M4	**Eigenvalues & Diagonalization** Eigenvalues and eigenvectors; characteristic polynomial; diagonalization of matrices; Cayley-Hamilton theorem and applications.	6
M5	**Inner Product Spaces & SVD** Inner product spaces; norms; orthogonality; Gram-Schmidt orthonormalization; Singular Value Decomposition (SVD) and applications.	5
	Total Contact Hours (Lecture)	**28**

Text Books

1. S. Lipschutz, M. Lipson, Schaum's Outlines of Linear Algebra, McGraw-Hill.
2. G. Strang, Introduction to Linear Algebra, 4th ed., Wellesley-Cambridge Press, 2009.

Reference Books

1. J. Defranza, D. Gagliardi, Introduction to Linear Algebra with Applications, McGraw-Hill.
2. K. Hoffman, R. Kunze, Linear Algebra, Prentice Hall.

Industry Relevance

Linear algebra is the computational core of every modern signal-processing, communication (MIMO, beamforming), machine-learning, computer-vision, and EDA workload. SVD and eigen-analysis are routinely used in channel estimation, data compression, PCA, and recommender systems across the telecom, semiconductor, and AI industries.

Open Educational Resources (OERs) & Open Repositories

- MIT OCW 18.06 Linear Algebra (Gilbert Strang) — https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/
- NPTEL: Linear Algebra — https://nptel.ac.in/courses/111106051
- 3Blue1Brown: Essence of Linear Algebra — https://www.3blue1brown.com/topics/linear-algebra
- Khan Academy: Linear Algebra — https://www.khanacademy.org/math/linear-algebra

Suggested Laboratory Exercises

1. Solve and visualize linear systems and row-reduction using Python/NumPy or GNU Octave.
2. Compute rank, inverse, and determinant for engineering matrices.
3. Eigen-decomposition and diagonalization of sample covariance matrices.
4. Image compression demonstration using SVD.

Open-Source Simulators / Tools

Python (NumPy, SciPy, Matplotlib) • GNU Octave • Sage • Jupyter

MA-L-02-C Calculus

Course Code	MA-L-02-C	**Credits**	3
Category	BSC (Basic Science)	**Structure (L-T-P-S)**	2-1-0-0
Prerequisite	Nil	**Laboratory**	No

Develops single- and multivariable calculus tools essential for modelling continuous systems, optimization, and field theory in engineering.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Find limits of functions graphically, numerically, and algebraically.
CO2	Apply continuity and differentiability to algebraic and transcendental functions.
CO3	Use partial derivatives, gradients, and Lagrange multipliers for optimization.
CO4	Apply the Fundamental Theorem of Calculus to evaluate definite integrals.
CO5	Evaluate multiple integrals to solve area, volume, and real-world problems.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Single-Variable Calculus Review** Limit, continuity and differentiability; indeterminate forms and L'Hospital rule; Mean Value theorem; Maclaurin and Taylor series.	5
M2	**Functions of Several Variables** Limits and continuity; partial derivatives and differentiability; linearization and differentials; chain rule; gradient vector; tangent planes; directional derivatives.	6
M3	**Multivariable Optimization** Extreme values and saddle points; Lagrange multipliers; Taylor's formula for several variables; partial derivatives with constrained variables.	5
M4	**Double Integrals** Double integrals; change of order of integration; area and volume by double integral; double integrals in polar form.	6
M5	**Triple Integrals** Triple integrals in rectangular, cylindrical, and spherical coordinates; substitutions in multiple integrals; applications.	6
	Total Contact Hours (Lecture)	**28**

Text Books

1. M. D. Weir, J. Hass, Thomas' Calculus, 12th ed., Pearson.
2. G. B. Thomas, R. L. Finney, Calculus and Analytic Geometry, 9th ed., Pearson.

Reference Books

1. Hughes-Hallett et al., Calculus: Single and Multivariable, 6th ed., Wiley.
2. J. Stewart, Multivariable Calculus, Hybrid Edition, Cengage.

Industry Relevance

Differential and integral calculus underlie modelling of analog circuits, electromagnetic fields, control loops, and optimization in ML training (gradient descent). Multivariable optimization with constraints is a daily tool in design and resource-allocation problems in communication and VLSI.

Open Educational Resources (OERs) & Open Repositories

- MIT OCW 18.01 / 18.02 Calculus — https://ocw.mit.edu/courses/mathematics/
- NPTEL: Calculus of Several Real Variables — https://nptel.ac.in/courses/111105134
- Paul's Online Math Notes — https://tutorial.math.lamar.edu/
- Khan Academy: Multivariable Calculus — https://www.khanacademy.org/math/multivariable-calculus

Suggested Laboratory Exercises

1. Symbolic differentiation and integration with SymPy / Maxima.
2. Visualization of gradients, tangent planes, and saddle points.
3. Numerical evaluation of double/triple integrals.
4. Constrained optimization using Lagrange multipliers (case studies).

Open-Source Simulators / Tools

Python (SymPy, NumPy, Matplotlib) • Maxima • GeoGebra • Desmos

CS-L-01-C Problem Solving with C Programming

Course Code	CS-L-01-C	**Credits**	3
Category	ESC / VSEC	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Nil	**Laboratory**	Mandatory (2 Hours)

Builds algorithmic problem-solving ability and procedural programming skills in C, the foundation for embedded and systems programming in electronics.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Identify and list the programming tasks involved in a computational problem.
CO2	Implement conditional branching, iteration, and recursion in C.
CO3	Use arrays, pointers, and structures to develop algorithms and programs.
CO4	Apply dynamic memory allocation and file handling in C.
CO5	Design modular C programs using functions and good coding practices.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Fundamentals of C** Computer organization; computer languages; algorithm and flowchart; features and structure of a C program; basic elements: data types, operators, expressions, I/O.	5
M2	**Control Flow** Conditional instructions; iterations (for, while, do-while); unconditional statements (break, continue, goto); switch-case.	5
M3	**Arrays, Strings & Functions** One- and multi-dimensional arrays; strings; functions, parameter passing, scope; recursion.	6
M4	**Pointers** Pointers; pointers with arrays and functions; pointer arithmetic; dynamic memory allocation (malloc, calloc, realloc, free).	6
M5	**Structures, Unions & Files** Structures and unions; nested structures; arrays of structures; file handling: text and binary files, file operations.	6
	Total Contact Hours (Lecture)	**28**

Text Books

1. B. W. Kernighan, D. M. Ritchie, The C Programming Language, 2nd ed., Prentice Hall (ISBN 978-0131103627).
2. B. S. Gottfried, Schaum's Outline of Programming with C, McGraw-Hill (ISBN 978-0070240353).

Reference Books

1. E. Balagurusamy, Programming in ANSI C, McGraw-Hill.
2. Y. Kanetkar, Let Us C, BPB Publications.

Industry Relevance

C remains the dominant language for firmware, device drivers, RTOS, and microcontroller programming across the embedded, automotive, and IoT industries. Strong C and pointer skills are a baseline expectation for embedded-systems and systems-software roles.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Problem Solving through Programming in C — https://nptel.ac.in/courses/106105171
- CS50 Introduction to Computer Science (Harvard) — https://cs50.harvard.edu/x/
- GeeksforGeeks C Programming — https://www.geeksforgeeks.org/c-programming-language/
- learn-c.org Interactive Tutorial — https://www.learn-c.org/

Suggested Laboratory Exercises

1. Programs on operators, type conversion, and formatted I/O.
2. Branching and looping: prime check, factorial, patterns.
3. Array and string manipulation; searching and sorting.
4. Recursion: Fibonacci, Tower of Hanoi, GCD.
5. Pointers and dynamic memory; linked structures.
6. Structures, unions, and file read/write applications.

Open-Source Simulators / Tools

GCC / Clang • Code::Blocks • VS Code • Online GDB • Valgrind

EC-L-01-C Digital Electronics using Verilog

Course Code	EC-L-01-C	**Credits**	3
Category	ESC (Engineering Science)	**Structure (L-T-P-S)**	2-0-2-0
Prerequisite	Nil	**Laboratory**	Mandatory (2 Hours)

Introduces digital logic design and hardware description using Verilog HDL, forming the gateway to VLSI, FPGA, and embedded design.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Understand number systems, logic gates, Boolean simplification, and flip-flops.
CO2	Design basic digital systems using standard ICs and Verilog.
CO3	Analyse and design combinational circuits.
CO4	Analyse and design sequential circuits including counters and FSMs.
CO5	Implement and simulate digital designs in Verilog HDL.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Number Systems & Boolean Algebra** Number systems and conversions; logic gates and truth tables; Boolean algebra; K-maps; NAND/NOR implementations; tabular (Quine-McCluskey) simplification.	5
M2	**Combinational Design - Arithmetic** Design procedure; half/full adder and subtractor; 4-bit ripple-carry adder; carry-look-ahead adder; BCD arithmetic.	6
M3	**Combinational Design - MSI** Decoders, encoders, multiplexers, de-multiplexers, magnitude comparators; implementing logic with MUX/decoders.	5
M4	**Sequential Circuits** Latches and flip-flops; FF conversions; serial and parallel shift registers; synchronous and asynchronous counter design.	6
M5	**FSM & Verilog HDL** Mealy and Moore machines; state diagrams and tables; Verilog modelling styles (gate, dataflow, behavioral); test benches and simulation.	6
	Total Contact Hours (Lecture)	**28**

Text Books

1. R. P. Jain, Modern Digital Electronics, 3rd ed., Tata McGraw-Hill, 2003.
2. M. M. Mano, M. D. Ciletti, Digital Design with an Introduction to the Verilog HDL, 5th ed., Pearson, 2013.

Reference Books

1. T. L. Floyd, Digital Fundamentals, 10th ed., Pearson, 2011.
2. S. Palnitkar, Verilog HDL, 2nd ed., Pearson.

Industry Relevance

Digital design and Verilog are the entry skills for the semiconductor and FPGA industry. RTL design and simulation skills built here are directly used in chip-design houses (Intel, Qualcomm, NXP, Synopsys, Cadence) and align with the India Semiconductor Mission.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Digital Circuits — https://nptel.ac.in/courses/108105132
- NPTEL: Hardware Modeling using Verilog — https://nptel.ac.in/courses/106105165
- EDA Playground (online HDL simulator) — https://www.edaplayground.com/
- nandland.com Verilog & FPGA tutorials — https://nandland.com/

Suggested Laboratory Exercises

1. Verify logic gates and De Morgan's theorems in Verilog.
2. Design and simulate adders/subtractors and a 4-bit ALU.
3. Implement MUX, decoder, encoder, and comparator.
4. Design flip-flops and shift registers.
5. Design synchronous and asynchronous counters.
6. FSM design: sequence detector / traffic-light controller.

Open-Source Simulators / Tools

Icarus Verilog + GTKWave • Verilator • EDA Playground • Xilinx Vivado (free WebPACK) • Logisim Evolution

EC-L-02-C Introduction to Internet of Things

Course Code	EC-L-02-C	**Credits**	2
Category	VSEC	**Structure (L-T-P-S)**	1-0-2-2
Prerequisite	Nil	**Laboratory**	Mandatory

Introduces IoT architecture, sensors, microcontrollers, connectivity, and cloud platforms with hands-on prototyping.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Understand sensors, actuators, and signal conditioning in measurement systems.
CO2	Describe IoT architecture and real-world design constraints.
CO3	Interface sensors with Arduino / ESP32 / Raspberry Pi.
CO4	Use IoT communication protocols and cloud platforms.
CO5	Design and prototype IoT solutions for real applications.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Introduction to IoT** IoT definition, history, and evolution; IoT vs traditional embedded systems; examples from smart cities, healthcare, agriculture, and industry.	3
M2	**IoT Architecture & Components** Layered architecture - perception, network, application; sensors, actuators, microcontrollers, communication modules.	3
M3	**Hardware Platforms** Arduino, ESP32/ESP8266, Raspberry Pi basics; development boards and their features; interfacing fundamentals.	3
M4	**Communication & Cloud** Wi-Fi, Bluetooth, Zigbee, LoRa, NB-IoT; choosing a protocol; MQTT; ThingSpeak, Blynk, Google/AWS IoT.	3
M5	**IoT Data & Security** Data collection, basic analytics and visualization; security challenges and good practices; mini-project launch.	2
	Total Contact Hours (Lecture)	**14**

Text Books

1. A. Bahga, V. Madisetti, Internet of Things: A Hands-On Approach, Universities Press.
2. J. Holler, V. Tsiatsis et al., From Machine-to-Machine to the Internet of Things, Elsevier.

Reference Books

1. R. Buyya, A. V. Dastjerdi, Internet of Things: Principles and Paradigms, Morgan Kaufmann.
2. Vendor documentation: ESP32, ThingSpeak, MQTT.

Industry Relevance

IoT skills feed directly into smart-manufacturing, automotive, agritech, and smart-city industries. Sensor interfacing, edge connectivity, and cloud dashboards are core to the rapidly growing IoT and Industry-4.0 job market.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Introduction to Internet of Things — https://nptel.ac.in/courses/106105166
- Wokwi IoT Simulator — https://wokwi.com/
- Tinkercad Circuits — https://www.tinkercad.com/circuits
- Hackster.io IoT Projects — https://www.hackster.io/projects

Suggested Laboratory Exercises

1. Blink and digital/analog I/O on Arduino/ESP32.
2. Read DHT11/LDR/PIR sensors and display values.
3. Log sensor data to ThingSpeak and visualize charts.
4. Relay/LED control via Blynk app (home automation).
5. Mini-project: smart plant watering / weather station.

Open-Source Simulators / Tools

Arduino IDE • Wokwi • Tinkercad Circuits • MQTT Explorer • ThingSpeak • Blynk

PH-L-01-C Quantum Mechanics

Course Code	PH-L-01-C	**Credits**	3
Category	BSC (Basic Science)	**Structure (L-T-P-S)**	2-1-0-0
Prerequisite	Nil	**Laboratory**	No

Introduces the principles and formalism of quantum mechanics that underpin modern electronics, photonics, semiconductor devices, and quantum technologies.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Use the language and formalism of basic quantum mechanics.
CO2	Carry out calculations for atoms and quantum phenomena in electron physics.
CO3	Interpret spectroscopic results in quantized units.
CO4	Solve the Schrödinger equation for standard potentials.
CO5	Relate quantum concepts to qubits and emerging quantum technologies.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Foundations** Basic principles; wave function and uncertainty principle; probability amplitude and density; Schrödinger formalism; time-dependent and time-independent equations; eigenstates, eigenvalues, expectation values.	6
M2	**Wave Mechanics in 1-D** Schrödinger equation in one dimension; particle in a box; potential step and barrier; tunnelling; harmonic oscillator.	6
M3	**Angular Momentum & Central Potential** Orbital and spin angular-momentum operators; eigenstates/eigenvalues of angular momentum; central potential.	6
M4	**Hydrogen Atom** Solution of the Schrödinger equation in a central potential; hydrogen-like atom; quantum numbers and applications.	5
M5	**Dirac Formalism & Qubits** Dirac bra-ket algebra; angular momenta; Clebsch-Gordon coefficients; spin-orbit interaction; Hilbert space and the concept of qubits.	5
	Total Contact Hours (Lecture)	**28**

Text Books

1. A. Ghatak, S. Lokanathan, Quantum Mechanics: Theory and Applications.
2. D. J. Griffiths, Introduction to Quantum Mechanics, 2nd ed., Pearson.

Reference Books

1. L. I. Schiff, Quantum Mechanics, McGraw-Hill.
2. R. Shankar, Principles of Quantum Mechanics, Springer.

Industry Relevance

Quantum mechanics underlies semiconductor physics, lasers, photonics, and the emerging quantum-computing and quantum-communication industries. It is foundational for device-physics and R&D roles in semiconductor and quantum-technology organizations.

Open Educational Resources (OERs) & Open Repositories

- MIT OCW 8.04 Quantum Physics I — https://ocw.mit.edu/courses/8-04-quantum-physics-i-spring-2016/
- NPTEL: Quantum Mechanics — https://nptel.ac.in/courses/115106066
- IBM Quantum Learning — https://learning.quantum.ibm.com/
- Qiskit Textbook — https://qiskit.org/learn/

Suggested Laboratory Exercises

1. Numerical solution of the 1-D Schrödinger equation (Python).
2. Visualization of particle-in-a-box wavefunctions and probability densities.
3. Tunnelling probability vs barrier width simulation.
4. Single-qubit states and gates on the Bloch sphere using Qiskit.

Open-Source Simulators / Tools

Python (NumPy, QuTiP, Matplotlib) • Qiskit • IBM Quantum Composer

MA-L-03-C Probability and Statistics

Course Code	MA-L-03-C	**Credits**	3
Category	BSC (Basic Science)	**Structure (L-T-P-S)**	2-1-0-0
Prerequisite	Nil	**Laboratory**	No

Provides probability and statistical inference tools essential for communication theory, signal processing, reliability, and machine learning.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Apply probability rules including additive/multiplicative laws and Bayes' rule.
CO2	Analyse data using measures of central tendency, dispersion, and location.
CO3	Identify and use discrete and continuous distributions.
CO4	Translate real-world problems into probability models.
CO5	Perform estimation and hypothesis testing.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Probability Foundations** Axiomatic definition; finite sample spaces; joint and conditional probability; independence; total probability; Bayes' rule and applications.	6
M2	**Random Variables** Discrete and continuous random variables; cdf, pmf, pdf and properties; bivariate and multivariate random variables; jointly distributed random variables; conditional and joint densities.	6
M3	**Distributions & CLT** Standard discrete and continuous distributions; sampling and sampling distributions; Central Limit Theorem.	6
M4	**Descriptive Statistics** Measures of central tendency, dispersion, skewness, kurtosis; data representation: histogram, pie chart, box-plot, bi-plot, multidimensional scaling.	5
M5	**Inference** Methods of estimation; unbiased estimators; confidence intervals (Z, t); hypothesis testing; types of errors; rejection-region and p-value approaches.	5
	Total Contact Hours (Lecture)	**28**

Text Books

1. A. J. Hayter, Probability and Statistics for Engineers, 4th ed., Cengage, 2012.
2. S. C. Gupta, V. K. Kapoor, Fundamentals of Mathematical Statistics, 11th ed., S. Chand, 2014.

Reference Books

1. S. M. Ross, Introduction to Probability and Statistics for Engineers and Scientists, 5th ed., Academic Press.

2. P. L. Meyer, Introductory Probability and Statistical Applications, Addison-Wesley.

Industry Relevance

Probability and statistics drive digital-communication theory (noise, detection, estimation), reliability engineering, A/B testing, and the entire machine-learning/data-science pipeline used across telecom, semiconductor, and analytics industries.

Open Educational Resources (OERs) & Open Repositories

- MIT OCW 6.041 Probabilistic Systems Analysis — https://ocw.mit.edu/courses/6-041-probabilistic-systems-analysis-and-applied-probability-fall-2010/
- NPTEL: Probability and Statistics — https://nptel.ac.in/courses/111105041
- Seeing Theory (visual probability) — https://seeing-theory.brown.edu/
- StatQuest (YouTube/Open) — https://statquest.org/

Suggested Laboratory Exercises

1. Simulate random experiments and verify probability laws (Python).
2. Fit and visualize standard distributions to data.
3. Demonstrate the Central Limit Theorem by sampling.
4. Confidence intervals and hypothesis tests on real datasets.

Open-Source Simulators / Tools

Python (NumPy, SciPy, statsmodels, seaborn) • R • Jamovi • Jupyter

CS-L-02-C Data Structures

Course Code	CS-L-02-C	**Credits**	3
Category	ESC / VSEC	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Problem Solving with C Programming	**Laboratory**	Mandatory

Covers fundamental data structures and their applications, essential for efficient algorithm and embedded-software design.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Implement arrays and structures to solve real-time problems.
CO2	Apply stacks and queues to relevant problems.
CO3	Develop solutions using variations of stacks and queues.
CO4	Manage data using linked-list variations.
CO5	Devise tree- and graph-based application solutions.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Stacks & Queues** Stacks; stacks using dynamic arrays; queues; circular queues; evaluation of expressions (infix/postfix).	6
M2	**Linked Lists** Singly, doubly, and circular linked lists; polynomials; additional list operations; applications.	6
M3	**Trees** Binary trees; tree traversals; additional binary-tree operations; heaps; binary search trees.	6
M4	**Graphs** Graph ADT; representations; elementary graph operations; BFS and DFS; spanning trees.	5
M5	**Searching, Sorting & Hashing** Linear and binary search; bubble, insertion, selection, quick, merge, heap sort; hashing and collision resolution.	5
	Total Contact Hours (Lecture)	**28**

Text Books

1. Horowitz, Sahni, Anderson-Freed, Fundamentals of Data Structures in C, 2nd ed., Universities Press, 2008.
2. Y. Langsam, M. Augenstein, A. Tannenbaum, Data Structures Using C and C++, 2nd ed., Pearson, 2003.

Reference Books

1. S. Lipschutz, G. A. V. Pai, Data Structures, Schaum's Outlines, McGraw-Hill, 2008.
2. R. F. Gilberg, B. A. Forouzan, Data Structures: A Pseudocode Approach with C, Cengage, 2005.

Industry Relevance

Efficient data structures are essential for embedded firmware, real-time systems, EDA tools, and any performance-critical software. Mastery of these is a universal screening criterion in technical interviews across the IT and core-electronics industries.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Data Structures and Algorithms — https://nptel.ac.in/courses/106102064
- VisuAlgo (algorithm visualization) — https://visualgo.net/
- GeeksforGeeks Data Structures — https://www.geeksforgeeks.org/data-structures/
- MIT OCW 6.006 Introduction to Algorithms — https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

Suggested Laboratory Exercises

1. Stack and queue implementation; expression evaluation.
2. Singly/doubly/circular linked-list operations.
3. Binary search tree and traversals.
4. Heap and priority-queue implementation.
5. Graph traversal (BFS/DFS) and shortest path.
6. Comparison of sorting algorithms and hashing.

Open-Source Simulators / Tools

GCC • VS Code • VisuAlgo • Python Tutor (visualization)

EC-L-03-C Signals and Systems

Course Code	EC-L-03-C	**Credits**	3
Category	ESC (Engineering Science)	**Structure (L-T-P-S)**	2-0-2-0
Prerequisite	Nil	**Laboratory**	Faculty Choice

Establishes the mathematical framework of continuous- and discrete-time signals and LTI systems, the bedrock of communication, DSP, and control.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Classify continuous and discrete signals by their properties.
CO2	Classify systems based on their properties.
CO3	Analyse continuous-time signals and systems with appropriate tools.
CO4	Analyse sampling and discrete-time signals.
CO5	Determine Fourier transforms for continuous- and discrete-time signals.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Signals & Systems Basics** Classification of signals; signal operations (scaling, shifting, inversion); signal properties; elementary signals; classification and properties of systems.	6
M2	**LTI Systems & Convolution** Continuous-time LTI and discrete-time LSI systems; impulse and step response; convolution; system representation by differential/difference equations; eigenfunctions; frequency response.	6
M3	**Fourier Analysis** Fourier series of CT and DT signals; continuous-time Fourier transform and properties; Parseval's relation; time-bandwidth product.	6
M4	**DTFT, DFT & Sampling** Discrete-time Fourier transform and properties; relations among Fourier representations; DFT; sampling theorem; aliasing; reconstruction (ideal interpolator, ZOH, FOH).	5
M5	**Transform-Domain Analysis** Laplace/Z-transform: definition, region of convergence, properties; transform-domain analysis of LTI/LSI systems; system function, poles and zeros; stability.	5
	Total Contact Hours (Lecture)	**28**

Text Books

1. A. V. Oppenheim, A. S. Willsky, H. S. Nawab, Signals and Systems, Prentice Hall of India, 2006.
2. B. P. Lathi, Signal Processing and Linear Systems, Oxford University Press, 1998.

Reference Books

1. M. J. Roberts, Fundamentals of Signals and Systems, Tata McGraw-Hill, 2007.

2. R. F. Ziemer, W. H. Tranter, D. R. Fannin, Signals and Systems - Continuous and Discrete, 4th ed., Prentice Hall, 1998.

Industry Relevance

Signals and systems is the conceptual backbone of communication, audio/image/biomedical processing, radar, and control. The Fourier/Laplace/Z toolkit is used daily in DSP, telecom, and embedded-product R&D.

Open Educational Resources (OERs) & Open Repositories

- MIT OCW 6.003 Signals and Systems — https://ocw.mit.edu/courses/6-003-signals-and-systems-fall-2011/
- NPTEL: Signals and Systems — https://nptel.ac.in/courses/108104100
- Julius Smith - DSP-related Online Books (CCRMA) — https://ccrma.stanford.edu/~jos/
- Khan Academy / interactive Fourier demos — https://www.jezzamon.com/fourier/

Suggested Laboratory Exercises

1. Generate and operate on elementary signals in Python/Octave.
2. Compute and plot convolution of CT/DT signals.
3. Fourier series synthesis and CTFT/DTFT visualization.
4. DFT/FFT of signals and spectral analysis.
5. Sampling and aliasing demonstration; signal reconstruction.
6. Pole-zero plots and stability analysis.

Open-Source Simulators / Tools

Python (NumPy, SciPy, Matplotlib) • GNU Octave • Scilab

DA-L-02-C Introduction to AI & ML

Course Code	DA-L-02-C	**Credits**	2
Category	ESC / VSEC	**Structure (L-T-P-S)**	1-0-2-2
Prerequisite	Linear Algebra and Probability	**Laboratory**	Mandatory

Provides an intuitive, application-oriented introduction to AI and machine-learning workflows, tools, and core algorithms.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Distinguish AI from machine learning and deep learning.
CO2	Appraise the mathematics behind ML and the supervised/unsupervised paradigms.
CO3	Decide which ML paradigm suits a given real-world application.
CO4	Build and evaluate basic ML models using standard tools.
CO5	Recognize ethical and responsible-AI considerations.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Introduction to AI & ML** History and evolution of AI; AI vs ML vs DL; real-world applications across sectors.	3
M2	**Data & Problem Formulation** Datasets, features, labels; types of ML problems - classification, regression, clustering; data preprocessing.	3
M3	**ML Workflow** Train/test split; model training; evaluation metrics - accuracy, precision, recall, F1; overfitting and validation.	3
M4	**Core Algorithms** Linear regression; decision trees; k-NN; Naïve Bayes; introduction to neural networks.	4
M5	**Tools & Responsible AI** Google Colab, Jupyter, Kaggle; bias, fairness, explainability, data privacy; competition launch.	1
	Total Contact Hours (Lecture)	**14**

Text Books

1. A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, O'Reilly.
2. S. Raschka, Python Machine Learning, Packt Publishing.

Reference Books

1. T. Mitchell, Machine Learning, McGraw-Hill.
2. C. M. Bishop, Pattern Recognition and Machine Learning, Springer.

Industry Relevance

ML is now embedded in virtually every electronics and telecom product - from AI-on-edge and intelligent communication to predictive maintenance. Familiarity with the ML workflow and tooling is a strong differentiator for ECE graduates entering data-driven roles.

Open Educational Resources (OERs) & Open Repositories

- Google ML Crash Course — https://developers.google.com/machine-learning/crash-course
- NPTEL: Introduction to Machine Learning — https://nptel.ac.in/courses/106106139
- Kaggle Learn Micro-Courses — https://www.kaggle.com/learn
- scikit-learn User Guide — https://scikit-learn.org/stable/user_guide.html

Suggested Laboratory Exercises

1. Data exploration and preprocessing with pandas.
2. Linear/logistic regression on a sample dataset.
3. Decision tree and k-NN classification.
4. Model evaluation with metrics and cross-validation.
5. Kaggle beginner competition (e.g., Titanic).

Open-Source Simulators / Tools

Python (scikit-learn, pandas, matplotlib, seaborn) • Google Colab • Jupyter • Kaggle

MA-L-04-C Discrete Mathematics

Course Code	MA-L-04-C	**Credits**	3
Category	BSC (Basic Science)	**Structure (L-T-P-S)**	2-1-0-0
Prerequisite	Basic set theory and logic	**Laboratory**	—

Provides the mathematical foundation for computer science: logic, sets, relations, combinatorics, graphs, and algebraic structures used throughout algorithms, databases, and theory.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Apply propositional and predicate logic to reason about statements and proofs.
CO2	Use sets, relations, and functions to model computational problems.
CO3	Apply counting, permutations, combinations, and recurrence relations.
CO4	Analyse graphs and trees and apply them to computing problems.
CO5	Explain basic algebraic structures relevant to computer science.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Logic & Proofs** Propositional logic; logical equivalences; predicates and quantifiers; rules of inference; methods of proof - direct, contradiction, induction.	7
M2	**Sets, Relations & Functions** Sets and operations; relations and their properties; equivalence relations; partial orders; functions; cardinality and countability.	7
M3	**Combinatorics** Counting principles; permutations and combinations; pigeonhole principle; binomial theorem; recurrence relations and generating functions.	7
M4	**Graph Theory** Graphs and terminology; paths and connectivity; Euler and Hamiltonian graphs; trees; spanning trees; graph colouring.	7
M5	**Algebraic Structures** Groups, subgroups, and homomorphisms; rings and fields (introduction); Boolean algebra; applications to coding and computing.	7
	Total Contact Hours (Lecture)	**35**

Text Books

1. K. H. Rosen, Discrete Mathematics and Its Applications, McGraw-Hill.
2. C. L. Liu, Elements of Discrete Mathematics, McGraw-Hill.

Reference Books

1. R. P. Grimaldi, Discrete and Combinatorial Mathematics, Pearson.
2. J. P. Tremblay, R. Manohar, Discrete Mathematical Structures with Applications to Computer Science, McGraw-Hill.

Industry Relevance

Discrete mathematics underpins algorithm design, cryptography, databases, formal verification, and network analysis. The reasoning and proof skills it develops are foundational for technical interviews and for advanced study in theoretical computer science and AI.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Discrete Mathematics — https://nptel.ac.in/courses/106106183
- MIT OCW 6.042J Mathematics for Computer Science — https://ocw.mit.edu/
- Open Logic Project (free textbook) — https://openlogicproject.org/
- SageMath (open computer algebra) — https://www.sagemath.org/

Suggested Laboratory Exercises

1. Truth-table and logical-equivalence verification in Python.
2. Relation properties and closure computation.
3. Counting and recurrence-solving exercises.
4. Graph representation and traversal.
5. Group/Boolean-algebra computations with SageMath.

Open-Source Simulators / Tools

Python (SymPy) • SageMath • GAP (groups) • Jupyter

CS-L-03-C Design and Analysis of Algorithms

Course Code	CS-L-03-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Data Structures, Discrete Mathematics	**Laboratory**	Mandatory

Covers the design paradigms and rigorous analysis of algorithms, including complexity analysis, classic algorithmic techniques, and intractability.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Analyse time and space complexity using asymptotic notation.
CO2	Design algorithms using divide-and-conquer and greedy strategies.
CO3	Apply dynamic programming to optimization problems.
CO4	Apply graph algorithms to computational problems.
CO5	Explain NP-completeness and approaches to intractable problems.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Foundations & Complexity** Algorithm analysis; asymptotic notation; recurrences; master theorem; correctness; amortized analysis.	7
M2	**Divide & Conquer** Merge sort; quicksort and analysis; binary search; Strassen's matrix multiplication; closest pair of points.	6
M3	**Greedy & Dynamic Programming** Greedy method; activity selection; Huffman coding; dynamic programming; LCS; matrix-chain multiplication; knapsack.	8
M4	**Graph Algorithms** BFS/DFS; topological sort; minimum spanning trees (Kruskal, Prim); shortest paths (Dijkstra, Bellman-Ford, Floyd-Warshall).	7
M5	**Complexity Classes** P, NP, NP-complete, NP-hard; reductions; Cook's theorem (overview); approximation and backtracking strategies.	6
	Total Contact Hours (Lecture)	**34**

Text Books

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Introduction to Algorithms, MIT Press.
2. J. Kleinberg, E. Tardos, Algorithm Design, Pearson.

Reference Books

1. S. Dasgupta, C. Papadimitriou, U. Vazirani, Algorithms, McGraw-Hill.
2. A. V. Aho, J. E. Hopcroft, J. D. Ullman, The Design and Analysis of Computer Algorithms, Pearson.

Industry Relevance

Algorithm design and complexity analysis are the single most tested skill in software-engineering interviews and are essential to building efficient, scalable systems. These competencies are core across product, systems, and research roles.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Design and Analysis of Algorithms — https://nptel.ac.in/courses/106106131
- MIT OCW 6.006 Introduction to Algorithms — https://ocw.mit.edu/
- VisuAlgo (algorithm visualisation) — https://visualgo.net/
- Open Data Structures (free book) — https://opendatastructures.org/

Suggested Laboratory Exercises

1. Implement and compare sorting algorithms.
2. Divide-and-conquer problem solving.
3. Dynamic-programming solutions (LCS, knapsack).
4. Graph algorithms (MST, shortest paths).
5. Backtracking/branch-and-bound exercise.

Open-Source Simulators / Tools

Python / C++ (GCC) • VisuAlgo • Jupyter • GDB

CS-L-04-C Database Management Systems

Course Code	CS-L-04-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Data Structures	**Laboratory**	Mandatory

Covers the design, querying, and internals of database systems, including the relational model, SQL, normalization, transactions, and storage.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Design conceptual and relational database schemas.
CO2	Formulate queries using relational algebra and SQL.
CO3	Apply normalization to design well-structured schemas.
CO4	Explain transaction management, concurrency, and recovery.
CO5	Describe indexing, storage, and query-processing internals.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Data Models & ER** Database concepts and architecture; ER and Enhanced-ER modelling; relational model; keys and constraints.	6
M2	**Relational Algebra & SQL** Relational algebra and calculus; SQL - DDL, DML, joins, nested queries, aggregation; views; triggers; stored procedures.	8
M3	**Database Design** Functional dependencies; normal forms (1NF-BCNF, 3NF); decomposition; denormalization trade-offs.	6
M4	**Transactions & Concurrency** ACID properties; serializability; concurrency control - locking, timestamp; deadlock; recovery and logging.	7
M5	**Storage & Indexing** File organization; B/B+ trees; hashing; query processing and optimization; introduction to NoSQL and distributed databases.	7
	Total Contact Hours (Lecture)	**34**

Text Books

1. A. Silberschatz, H. F. Korth, S. Sudarshan, Database System Concepts, McGraw-Hill.
2. R. Elmasri, S. B. Navathe, Fundamentals of Database Systems, Pearson.

Reference Books

1. R. Ramakrishnan, J. Gehrke, Database Management Systems, McGraw-Hill.
2. C. J. Date, An Introduction to Database Systems, Pearson.

Industry Relevance

Database skills are universal in backend, data-engineering, and full-stack roles. SQL, schema design, and an understanding of transactions and indexing are essential for building reliable data-driven applications.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Database Management System — https://nptel.ac.in/courses/106105175
- CMU 15-445 Database Systems (open) — https://15445.courses.cs.cmu.edu/
- PostgreSQL official documentation — https://www.postgresql.org/docs/
- SQLBolt interactive SQL tutorial — https://sqlbolt.com/

Suggested Laboratory Exercises

1. ER design and schema creation.
2. Complex SQL queries with joins and subqueries.
3. Normalization and functional-dependency exercises.
4. Transactions, triggers, and stored procedures.
5. Indexing and query-plan analysis (EXPLAIN).

Open-Source Simulators / Tools

PostgreSQL • MySQL / MariaDB • SQLite • DBeaver • MongoDB (NoSQL intro)

CS-L-05-C Operating Systems

Course Code	CS-L-05-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Data Structures, Computer Organization (concurrent)	**Laboratory**	Mandatory

Covers the principles and internals of modern operating systems: processes, scheduling, memory management, concurrency, file systems, and I/O.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Explain operating-system structure, processes, and threads.
CO2	Apply CPU-scheduling and synchronization techniques.
CO3	Analyse and prevent deadlocks.
CO4	Explain memory-management and virtual-memory schemes.
CO5	Describe file-system and I/O subsystem design.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**OS Structure & Processes** OS functions and structures; system calls; processes and threads; process control block; context switching.	6
M2	**Scheduling & Synchronization** CPU scheduling algorithms; the critical-section problem; semaphores, mutexes, monitors; classical synchronization problems.	8
M3	**Deadlocks** Deadlock characterization; prevention, avoidance (banker's algorithm), detection, and recovery.	5
M4	**Memory Management** Contiguous allocation; paging; segmentation; virtual memory; demand paging; page replacement; thrashing.	8
M5	**File Systems & I/O** File concepts and directory structures; allocation methods; disk scheduling; I/O systems; introduction to protection and security.	7
	Total Contact Hours (Lecture)	**34**

Text Books

1. A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, Wiley.
2. A. S. Tanenbaum, H. Bos, Modern Operating Systems, Pearson.

Reference Books

1. W. Stallings, Operating Systems: Internals and Design Principles, Pearson.
2. R. Arpaci-Dusseau, A. Arpaci-Dusseau, Operating Systems: Three Easy Pieces (free).

Industry Relevance

Operating-system concepts underpin systems programming, backend performance, cloud infrastructure, and embedded development. Understanding processes, concurrency, and memory is essential for systems and infrastructure-engineering roles.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Introduction to Operating Systems — https://nptel.ac.in/courses/106106144
- Operating Systems: Three Easy Pieces (free book) — https://pages.cs.wisc.edu/~remzi/OSTEP/
- MIT OCW 6.828 / xv6 teaching OS — https://pdos.csail.mit.edu/6.828/
- The Linux Kernel documentation — https://www.kernel.org/doc/html/latest/

Suggested Laboratory Exercises

1. Process creation and inter-process communication.
2. CPU-scheduling algorithm simulation.
3. Producer-consumer with semaphores/threads.
4. Banker's deadlock-avoidance algorithm.
5. Page-replacement and disk-scheduling simulation.

Open-Source Simulators / Tools

GCC + POSIX threads • Linux • xv6 • GDB • QEMU

CS-L-06-C Object Oriented Programming

Course Code	CS-L-06-C	**Credits**	2
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	1-0-2-2
Prerequisite	Problem Solving with C Programming	**Laboratory**	Mandatory

Develops object-oriented design and programming skills using classes, inheritance, polymorphism, and design principles with a modern OOP language.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Apply object-oriented concepts of classes and objects.
CO2	Use inheritance and polymorphism to build reusable code.
CO3	Apply exception handling and generics/templates.
CO4	Use collections and the standard library effectively.
CO5	Apply basic design principles and patterns.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**OOP Fundamentals** Procedural vs object-oriented paradigms; classes and objects; encapsulation; constructors/destructors; access modifiers.	5
M2	**Inheritance & Polymorphism** Single and multiple inheritance; method overriding and overloading; virtual functions; dynamic dispatch; abstract classes and interfaces.	6
M3	**Exceptions & Generics** Exception handling; templates/generics; operator overloading; type casting.	5
M4	**Collections & I/O** Standard collections/containers; iterators; file and stream I/O; introduction to functional features (lambdas).	6
M5	**Design Principles** SOLID principles; common design patterns (singleton, factory, observer); introduction to UML; testing and debugging OOP code.	6
	Total Contact Hours (Lecture)	**28**

Text Books

1. E. Balagurusamy, Object Oriented Programming with C++, McGraw-Hill.
2. H. Schildt, Java: The Complete Reference, McGraw-Hill.

Reference Books

1. B. Stroustrup, The C++ Programming Language, Addison-Wesley.
2. E. Gamma et al., Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley.

Industry Relevance

Object-oriented programming is the dominant paradigm for large-scale application development. Mastery of OOP, design principles, and patterns is essential for software-engineering and backend roles across the industry.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Programming in Java — https://nptel.ac.in/courses/106105191
- MIT OCW 6.005 / 6.031 Software Construction — https://ocw.mit.edu/
- Refactoring.Guru design patterns — https://refactoring.guru/design-patterns
- cppreference.com — https://en.cppreference.com/

Suggested Laboratory Exercises

1. Class design with encapsulation.
2. Inheritance and polymorphism exercises.
3. Exception handling and generics/templates.
4. Collection and file-I/O programs.
5. Implement a small project applying design patterns.

Open-Source Simulators / Tools

C++ (GCC) • OpenJDK (Java) • VS Code • Git

MA-L-05-C Optimization Techniques

Course Code	MA-L-05-C	**Credits**	2
Category	PSC (Programme Specific Core)	**Structure (L-T-P-S)**	2-0-0-0
Prerequisite	Linear Algebra, Calculus	**Laboratory**	—

Introduces optimization formulations and methods used in machine learning, operations research, and engineering design.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Formulate problems as optimization models.
CO2	Solve linear programming problems and interpret duality.
CO3	Apply unconstrained optimization methods.
CO4	Apply constrained optimization and KKT conditions.
CO5	Apply optimization techniques to machine-learning problems.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Introduction & LP** Optimization problem classification; convex sets and functions; linear programming; simplex method; graphical interpretation.	5
M2	**Duality & Sensitivity** LP duality; sensitivity analysis; transportation and assignment problems.	5
M3	**Unconstrained Optimization** Gradient descent; Newton and quasi-Newton methods; line search; convergence.	5
M4	**Constrained Optimization** Lagrange multipliers; KKT conditions; penalty and barrier methods; quadratic programming.	5
M5	**Optimization for ML** Stochastic gradient descent; momentum and adaptive methods; convex optimization in machine learning.	4
	Total Contact Hours (Lecture)	**24**

Text Books

1. S. S. Rao, Engineering Optimization: Theory and Practice, Wiley.
2. S. Boyd, L. Vandenberghe, Convex Optimization, Cambridge University Press.

Reference Books

1. E. K. P. Chong, S. H. Zak, An Introduction to Optimization, Wiley.
2. H. A. Taha, Operations Research: An Introduction, Pearson.

Industry Relevance

Optimization is the engine behind machine learning, operations research, and resource allocation. Gradient-based and convex-optimization skills are essential for ML, data-science, and quantitative-engineering roles.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Optimization Techniques — https://nptel.ac.in/courses/111105100
- Boyd Convex Optimization (Stanford) — https://web.stanford.edu/~boyd/cvxbook/
- CVXPY documentation — https://www.cvxpy.org/
- SciPy optimize reference — https://docs.scipy.org/doc/scipy/reference/optimize.html

Suggested Laboratory Exercises

1. LP solving with SciPy/PuLP.
2. Gradient descent and Newton's method.
3. Constrained optimization with KKT.
4. Convex modelling with CVXPY.
5. SGD for a small ML model.

Open-Source Simulators / Tools

Python (SciPy, PuLP, CVXPY) • GNU Octave • Jupyter

EC-L-05-C Digital Communication Systems

Course Code	EC-L-05-C	**Credits**	2
Category	PSC (Programme Specific Core)	**Structure (L-T-P-S)**	1-0-2-0
Prerequisite	Signals and Systems, Probability & Statistics	**Laboratory**	Mandatory

Introduces analog and digital communication principles relevant to networking, wireless, and data-transmission systems.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Design the digital modulation technique.
CO2	Design different line coding techniques.
CO3	Explain digital modulation and baseband transmission.
CO4	Describe sampling, quantization, and pulse-code modulation.
CO5	Explain the basics of information theory and multiplexing.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	Analog to digital conversion, Sampling theorem, Nyquist rate, Pulse code modulation (PCM), DPCM, Delta Modulation, ADM, A-law, Line coding, Digital modulation/demodulation techniques: ASK, BPSK, QPSK.	10
M2	MPSK, QPSK, MQAM, The Additive White Gaussian Noise (AWGN) Channel, Matched Filter detection, Error Probability for the AWGN Channel for different modulation, BER vs SNR plot, Constellation diagram, Inter-Symbol Interference	8
M3	Information theory, Source Coding Theorem, Huffman coding, Channel Coding Theorem, Shannon limit, Properties of Channel Capacity, Error control coding, Block coding, Convolution coding, BCH coding, Viterbi decoding	10
	Total Contact Hours (Lecture)	**28**

Text Books

1. M F Mesiya, “Contemporary Communication Systems,” Indian edition, McGraw Hill.
2. R. Mahapatra, “Analog and Digital Communication,” BPB publication 2024.

Reference Books

1. B. P. Lathi, "Modern Digital and Analog Communication systems,” Oxford University, 3rd Edition.
2. T. L. Singal, Analog and Digital Communications, McGraw-Hill.

Industry Relevance

Communication-systems fundamentals support networking, wireless, and IoT roles. Understanding modulation, noise, and channel capacity helps CSE engineers reason about data transmission and network performance.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Principles of Communication Systems — https://nptel.ac.in/courses/108104091
- MIT OCW 6.02 Introduction to EECS II — https://ocw.mit.edu/
- GNU Radio tutorials — https://wiki.gnuradio.org/
- SciPy signal-processing reference — https://docs.scipy.org/doc/scipy/reference/signal.html

Suggested Laboratory Exercises

- Nyquist rate, Quantization
- Digital Modulation
- QPSK modulation/Modulation
- Constellation diagram, BER vs SNR plot
- Error control coding

Open-Source Simulators / Tools

Python (NumPy, SciPy) • GNU Octave • GNU Radio

MA-L-06-C Graph Theory

Course Code	MA-L-06-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Discrete Mathematics	**Laboratory**	Mandatory

A deeper treatment of graph theory and its algorithms, central to networks, optimization, and many computing problems.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Apply graph terminology, representations, and properties.
CO2	Analyse connectivity, trees, and spanning structures.
CO3	Apply matching, covering, and colouring concepts.
CO4	Analyse planar graphs and network flows.
CO5	Apply graph algorithms to real computing problems.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Graph Fundamentals** Graphs, digraphs, and representations; degree sequences; isomorphism; walks, paths, cycles; connectivity.	6
M2	**Trees & Connectivity** Trees and their properties; spanning trees; minimum spanning trees; cut vertices and edges; vertex/edge connectivity.	7
M3	**Matching & Colouring** Bipartite graphs; matchings; Hall's theorem; vertex and edge colouring; chromatic number; independent and dominating sets.	7
M4	**Planarity & Flows** Planar graphs; Euler's formula; Kuratowski's theorem; network flows; max-flow min-cut theorem.	7
M5	**Advanced Topics** Eulerian and Hamiltonian graphs; spectral graph theory (introduction); random graphs; applications in social and computer networks.	8
	Total Contact Hours (Lecture)	**35**

Text Books

1. D. B. West, Introduction to Graph Theory, Pearson.
2. N. Deo, Graph Theory with Applications to Engineering and Computer Science, PHI.

Reference Books

1. R. Diestel, Graph Theory, Springer.
2. J. A. Bondy, U. S. R. Murty, Graph Theory, Springer.

Industry Relevance

Graph theory powers network routing, social-network analysis, recommendation systems, scheduling, and many optimization problems. Graph-algorithm skills are valuable across AI, networking, and large-scale systems roles.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Graph Theory — https://nptel.ac.in/courses/111106102
- MIT OCW 6.042J (graph theory units) — https://ocw.mit.edu/
- NetworkX documentation — https://networkx.org/
- Stanford SNAP datasets & library — https://snap.stanford.edu/

Suggested Laboratory Exercises

1. Graph representation and traversal.
2. Minimum spanning tree algorithms.
3. Bipartite matching implementation.
4. Graph colouring heuristics.
5. Max-flow computation and analysis.

Open-Source Simulators / Tools

Python (NetworkX) • Gephi • Jupyter • C++ (GCC)

CS-L-07-C Computer Networks

Course Code	CS-L-07-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Operating Systems (concurrent), Communication Systems	**Laboratory**	Mandatory

Covers the architecture, protocols, and algorithms of modern computer networks across the protocol stack, with hands-on socket and tool exercises.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Explain layered network architectures and reference models.
CO2	Analyse data-link layer protocols and error/flow control.
CO3	Apply IP addressing, routing, and forwarding concepts.
CO4	Explain transport-layer protocols (TCP/UDP) and congestion control.
CO5	Describe application-layer protocols and network security basics.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Introduction & Physical/Link** Network types and topologies; OSI and TCP/IP models; transmission media; framing; error detection/correction; MAC and Ethernet.	7
M2	**Data Link & LANs** Flow and error control; sliding window; ARQ; switching; VLANs; wireless LAN (802.11) basics.	6
M3	**Network Layer** IPv4/IPv6 addressing; subnetting; routing algorithms (distance-vector, link-state); OSPF, BGP basics; NAT and ICMP.	8
M4	**Transport Layer** UDP and TCP; connection management; reliability; flow and congestion control; sockets.	7
M5	**Application & Security** DNS, HTTP, SMTP, FTP; the web and CDNs; introduction to network security, TLS, and firewalls.	7
	Total Contact Hours (Lecture)	**35**

Text Books

1. J. F. Kurose, K. W. Ross, Computer Networking: A Top-Down Approach, Pearson.
2. A. S. Tanenbaum, D. J. Wetherall, Computer Networks, Pearson.

Reference Books

1. B. A. Forouzan, Data Communications and Networking, McGraw-Hill.
2. W. Stallings, Data and Computer Communications, Pearson.

Industry Relevance

Networking knowledge is essential for backend, cloud, DevOps, and security roles. Understanding TCP/IP, routing, and protocols is foundational for building and operating distributed and internet-scale systems.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Computer Networks and Internet Protocol — https://nptel.ac.in/courses/106105183
- Stanford CS144 Computer Networking (open) — https://cs144.github.io/
- Wireshark documentation & captures — https://www.wireshark.org/docs/
- Mininet network emulator — http://mininet.org/

Suggested Laboratory Exercises

1. Socket programming (TCP/UDP).
2. Packet capture and analysis with Wireshark.
3. Subnetting and IP-configuration exercises.
4. Routing simulation in Mininet/ns-3.
5. HTTP/DNS client implementation.

Open-Source Simulators / Tools

Wireshark • Mininet • ns-3 • Python sockets • GNS3

DA-L-03-C Computer Graphics and Visualization

Course Code	DA-L-03-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Linear Algebra, Data Structures	**Laboratory**	Mandatory

Covers the mathematics and algorithms of 2D/3D computer graphics, rendering, and data visualization with hands-on graphics programming.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Apply graphics primitives and 2D transformations.
CO2	Apply 3D transformations, projections, and viewing.
CO3	Implement rendering and clipping algorithms.
CO4	Apply illumination, shading, and texturing.
CO5	Apply visualization techniques to data.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Graphics Foundations** Graphics pipeline; raster vs vector; line and circle drawing (Bresenham); polygon filling; 2D transformations.	6
M2	**3D Transformations & Viewing** 3D transformations; homogeneous coordinates; projections (parallel, perspective); viewing pipeline; camera models.	7
M3	**Rendering & Clipping** Clipping algorithms; hidden-surface removal (z-buffer, painter's); rasterization; introduction to ray tracing.	7
M4	**Shading & Texturing** Illumination models; Gouraud and Phong shading; texture mapping; colour models; anti-aliasing.	7
M5	**Visualization** Data and information visualization; scientific and volume visualization; interaction; tools and libraries; GPU/shader basics.	7
	Total Contact Hours (Lecture)	**34**

Text Books

1. D. Hearn, M. P. Baker, Computer Graphics with OpenGL, Pearson.
2. J. D. Foley et al., Computer Graphics: Principles and Practice, Addison-Wesley.

Reference Books

1. E. Angel, D. Shreiner, Interactive Computer Graphics: A Top-Down Approach with WebGL, Pearson.
2. C. Ware, Information Visualization: Perception for Design, Morgan Kaufmann.

Industry Relevance

Graphics and visualization skills are used in gaming, AR/VR, simulation, scientific computing, and data-analytics dashboards. They support roles in graphics engineering, visualization, and front-end/data-presentation work.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Computer Graphics — https://nptel.ac.in/courses/106106090
- LearnOpenGL (open tutorials) — https://learnopengl.com/
- Three.js documentation — https://threejs.org/docs/
- D3.js data-visualization gallery — https://d3js.org/

Suggested Laboratory Exercises

1. Line/circle drawing and polygon fill.
2. 2D/3D transformation programs.
3. Clipping and hidden-surface removal.
4. Shading and texture mapping in OpenGL/WebGL.
5. Interactive data visualization with D3.js.

Open-Source Simulators / Tools

OpenGL / WebGL • Three.js • D3.js • Blender • Python (Matplotlib, Plotly)

CS-L-08-C Embedded Systems Design

Course Code	CS-L-08-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-0
Prerequisite	Digital Electronics, C Programming	**Laboratory**	Mandatory

Covers microcontroller-based embedded design, peripherals, interfacing, and real-time firmware for connected and intelligent devices.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Explain embedded-system architecture and design metrics.
CO2	Program microcontrollers and configure peripherals.
CO3	Interface sensors, actuators, and communication peripherals.
CO4	Apply interrupts, timers, and real-time concepts.
CO5	Design and debug a complete embedded application.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Embedded Fundamentals** Embedded-system characteristics; microprocessor vs microcontroller; ARM Cortex-M architecture; memory and bus organization; toolchain.	6
M2	**I/O & Peripherals** GPIO; timers/counters; PWM; ADC/DAC; watchdog; clock and power management.	6
M3	**Interrupts & Communication** Interrupt handling and priorities; serial protocols - UART, SPI, I2C; introduction to CAN.	6
M4	**RTOS Concepts** Polling vs interrupt-driven design; tasks, scheduling, semaphores, queues; introduction to FreeRTOS.	6
M5	**System Design** Sensor/actuator interfacing; low-power design; debugging and testing; IoT-connected embedded case studies.	6
	Total Contact Hours (Lecture)	**30**

Text Books

1. F. Vahid, T. Givargis, Embedded System Design: A Unified Hardware/Software Introduction, Wiley.
2. J. Yiu, The Definitive Guide to ARM Cortex-M3/M4 Processors, Elsevier.

Reference Books

1. R. Kamal, Embedded Systems: Architecture, Programming and Design, McGraw-Hill.
2. M. Mazidi et al., The STM32/ARM Microcontroller and Embedded Systems, Pearson.

Industry Relevance

Embedded skills are valuable in IoT, automotive, robotics, and consumer-electronics product teams. Firmware, RTOS, and peripheral interfacing are core to embedded-software roles that bridge hardware and software.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Embedded Systems Design — https://nptel.ac.in/courses/108102169
- ARM mbed / STM32 documentation — https://os.mbed.com/docs/
- FreeRTOS documentation — https://www.freertos.org/Documentation/RTOS_book.html
- Wokwi online microcontroller simulator — https://wokwi.com/

Suggested Laboratory Exercises

1. GPIO, timer, and PWM programming.
2. ADC and sensor interfacing.
3. UART/SPI/I2C communication.
4. Interrupt-driven design.
5. FreeRTOS multi-task application.

Open-Source Simulators / Tools

Arduino IDE / PlatformIO • STM32CubeIDE • Wokwi • FreeRTOS

CS-L-09-C Applied Machine Learning

Course Code	CS-L-09-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-0
Prerequisite	Probability & Statistics, Linear Algebra, Introduction to AIML	**Laboratory**	Mandatory

Covers practical machine-learning workflows, core supervised and unsupervised algorithms, model evaluation, and applied data pipelines.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Apply the end-to-end ML workflow and data preprocessing.
CO2	Build and evaluate supervised-learning models.
CO3	Apply unsupervised learning and dimensionality reduction.
CO4	Apply model selection, regularization, and validation.
CO5	Build applied ML pipelines for real datasets.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**ML Foundations & Data** Types of learning; data preprocessing; feature engineering; train/validation/test split; bias-variance trade-off.	6
M2	**Supervised Learning I** Linear and logistic regression; k-NN; decision trees; evaluation metrics (accuracy, precision/recall, ROC).	7
M3	**Supervised Learning II** Support vector machines; ensemble methods - bagging, random forests, boosting (XGBoost); naive Bayes.	7
M4	**Unsupervised Learning** Clustering (k-means, hierarchical, DBSCAN); dimensionality reduction (PCA, t-SNE); anomaly detection.	6
M5	**Applied ML & MLOps** Hyperparameter tuning; cross-validation; pipelines; model deployment basics; introduction to MLOps and responsible AI.	6
	Total Contact Hours (Lecture)	**32**

Text Books

1. A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow, O'Reilly.
2. C. M. Bishop, Pattern Recognition and Machine Learning, Springer.

Reference Books

1. T. Hastie, R. Tibshirani, J. Friedman, The Elements of Statistical Learning, Springer.
2. K. P. Murphy, Machine Learning: A Probabilistic Perspective, MIT Press.

Industry Relevance

Applied ML is among the most in-demand skill sets, powering recommendation, prediction, fraud-detection, and analytics products across every sector. These skills map directly to ML-engineer and data-scientist roles.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Machine Learning — https://nptel.ac.in/courses/106106139
- Andrew Ng / Stanford CS229 materials — https://cs229.stanford.edu/
- scikit-learn documentation & tutorials — https://scikit-learn.org/stable/
- Kaggle Learn (free micro-courses) — https://www.kaggle.com/learn

Suggested Laboratory Exercises

1. Data preprocessing and feature engineering.
2. Regression and classification with scikit-learn.
3. Ensemble methods and model comparison.
4. Clustering and PCA on a dataset.
5. End-to-end ML pipeline with tuning.

Open-Source Simulators / Tools

Python (scikit-learn, pandas, NumPy) • Jupyter • Google Colab • XGBoost

CS-L-10-C Finite Automata and Formal Languages

Course Code	CS-L-10-C	**Credits**	2
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-0-0
Prerequisite	Discrete Mathematics	**Laboratory**	—

Introduces the theory of computation: automata, formal languages, grammars, and the limits of computability.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Design finite automata and regular expressions for languages.
CO2	Apply properties of regular languages.
CO3	Design context-free grammars and pushdown automata.
CO4	Explain Turing machines and computability.
CO5	Explain decidability and the limits of computation.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Finite Automata** Alphabets, strings, languages; DFA and NFA; equivalence; epsilon-NFA; minimization.	6
M2	**Regular Languages** Regular expressions; Kleene's theorem; pumping lemma; closure properties; applications to lexical analysis.	5
M3	**Context-Free Languages** Context-free grammars; derivations and parse trees; ambiguity; normal forms (CNF, GNF); pushdown automata.	6
M4	**Turing Machines** Turing-machine model; variants; Church-Turing thesis; recursively enumerable and recursive languages.	5
M5	**Decidability** Decidable and undecidable problems; the halting problem; reducibility; introduction to complexity (P vs NP).	4
	Total Contact Hours (Lecture)	**26**

Text Books

1. J. E. Hopcroft, R. Motwani, J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, Pearson.
2. M. Sipser, Introduction to the Theory of Computation, Cengage.

Reference Books

1. P. Linz, An Introduction to Formal Languages and Automata, Jones & Bartlett.
2. K. L. P. Mishra, N. Chandrasekaran, Theory of Computer Science, PHI.

Industry Relevance

Automata and formal-language theory underpin compilers, parsers, regex engines, and language design. The abstraction and reasoning skills are foundational for systems, programming-language, and research roles.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Theory of Computation — https://nptel.ac.in/courses/106106049
- MIT OCW 18.404J Theory of Computation — https://ocw.mit.edu/
- JFLAP automata tool — https://www.jflap.org/
- Stanford Automata (open course materials) — https://online.stanford.edu/

Suggested Laboratory Exercises

1. DFA/NFA design and simulation in JFLAP.
2. Regular-expression to automata conversion.
3. Context-free grammar and parse-tree exercises.
4. Pushdown-automata design.
5. Turing-machine simulation.

Open-Source Simulators / Tools

JFLAP • Python • Automata libraries

CS-L-11-C Computer Organization and Architecture

Course Code	CS-L-11-C	**Credits**	2
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-0-0
Prerequisite	Digital Electronics using Verilog	**Laboratory**	—

Covers the organization and design of computer systems: instruction sets, processor datapaths, memory hierarchy, and performance.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Explain instruction-set architecture and machine instructions.
CO2	Analyse processor datapath and control design.
CO3	Apply pipelining and hazard-handling concepts.
CO4	Analyse memory hierarchy and cache performance.
CO5	Explain I/O organization and parallel architectures.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**ISA & Arithmetic** Computer abstractions; performance metrics; instruction-set architecture; addressing modes; integer and floating-point arithmetic.	5
M2	**Processor Design** Datapath and control; single-cycle and multi-cycle implementations; hardwired vs microprogrammed control.	5
M3	**Pipelining** Pipelined datapath; structural, data, and control hazards; forwarding; branch prediction; introduction to ILP.	5
M4	**Memory Hierarchy** Memory technologies; cache organization and mapping; cache performance; virtual memory and TLB.	5
M5	**I/O & Parallelism** I/O techniques; interrupts and DMA; buses; introduction to multicore, GPUs, and parallel architectures.	4
	Total Contact Hours (Lecture)	**24**

Text Books

1. D. A. Patterson, J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, Morgan Kaufmann.
2. C. Hamacher, Z. Vranesic, S. Zaky, Computer Organization, McGraw-Hill.

Reference Books

1. W. Stallings, Computer Organization and Architecture, Pearson.

2. J. L. Hennessy, D. A. Patterson, Computer Architecture: A Quantitative Approach, Morgan Kaufmann.

Industry Relevance

Computer-architecture knowledge informs performance optimization, systems programming, and hardware-software co-design. It is valuable for systems, embedded, and performance-engineering roles, and for writing efficient code.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Computer Organization and Architecture — https://nptel.ac.in/courses/106105163
- MIT OCW 6.004 Computation Structures — https://ocw.mit.edu/
- Ripes RISC-V visual simulator — https://github.com/mortbopet/Ripes
- RISC-V open ISA specifications — https://riscv.org/technical/specifications/

Suggested Laboratory Exercises

1. Assembly programming (MIPS/RISC-V).
2. Datapath and control simulation.
3. Pipeline hazard analysis.
4. Cache-performance simulation.
5. Performance benchmarking exercise.

Open-Source Simulators / Tools

Ripes (RISC-V) • MARS / SPIM (MIPS) • gem5 (intro) • Logisim

CS-L-13-C Cloud Computing

Course Code	CS-L-13-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Operating Systems, Computer Networks	**Laboratory**	Mandatory

Covers cloud service and deployment models, virtualization, distributed storage, and the design of scalable cloud-native applications.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Explain cloud service and deployment models.
CO2	Apply virtualization and containerization technologies.
CO3	Design scalable cloud storage and compute solutions.
CO4	Deploy and manage applications on cloud platforms.
CO5	Explain cloud security, cost, and reliability considerations.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Cloud Foundations** Evolution of cloud; service models (IaaS, PaaS, SaaS); deployment models; cloud economics; major providers.	6
M2	**Virtualization & Containers** Hypervisors; virtual machines; containers and Docker; orchestration with Kubernetes; serverless computing.	7
M3	**Cloud Storage & Data** Distributed storage; object/block/file storage; replication and consistency; databases in the cloud; CDNs.	6
M4	**Scalability & Services** Load balancing; auto-scaling; microservices; message queues; cloud-native design patterns; DevOps and CI/CD.	7
M5	**Security & Reliability** Identity and access management; cloud security; fault tolerance; monitoring; cost optimization; SLAs.	6
	Total Contact Hours (Lecture)	**32**

Text Books

1. T. Erl, R. Puttini, Z. Mahmood, Cloud Computing: Concepts, Technology & Architecture, Pearson.
2. R. Buyya, C. Vecchiola, S. T. Selvi, Mastering Cloud Computing, McGraw-Hill.

Reference Books

1. B. Wilder, Cloud Architecture Patterns, O'Reilly.
2. J. Arundel, J. Domingus, Cloud Native DevOps with Kubernetes, O'Reilly.

Industry Relevance

Cloud computing is foundational to virtually all modern software delivery. Skills in containers, Kubernetes, and cloud-native design map directly to cloud-engineer, DevOps, and backend roles at firms using AWS, Azure, and GCP.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Cloud Computing — https://nptel.ac.in/courses/106105167
- Kubernetes official documentation — https://kubernetes.io/docs/home/
- Docker getting-started guide — https://docs.docker.com/get-started/
- AWS / Azure free learning paths — https://www.cloudskillsboost.google/

Suggested Laboratory Exercises

1. Deploy a VM and configure cloud services.
2. Containerize an app with Docker.
3. Multi-container orchestration with Kubernetes.
4. Auto-scaling and load-balancing exercise.
5. Serverless function deployment.

Open-Source Simulators / Tools

Docker • Kubernetes (minikube) • OpenStack • Terraform • AWS/GCP free tier

CS-L-14-C Cryptography

Course Code	CS-L-14-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Discrete Mathematics, Probability & Statistics	**Laboratory**	Mandatory

Covers the mathematical foundations and practical algorithms of modern cryptography, including symmetric and public-key systems and their applications.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Explain classical and modern cryptographic principles.
CO2	Apply symmetric-key encryption algorithms.
CO3	Apply public-key cryptography and key exchange.
CO4	Apply hash functions and digital signatures.
CO5	Analyse cryptographic protocols and their security.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Foundations** Security goals; classical ciphers; number theory for cryptography - modular arithmetic, GCD, primes, Euler/Fermat.	6
M2	**Symmetric Cryptography** Stream and block ciphers; DES and AES; modes of operation; key management.	7
M3	**Public-Key Cryptography** RSA; Diffie-Hellman key exchange; elliptic-curve cryptography; the discrete-logarithm problem.	7
M4	**Integrity & Authentication** Hash functions (SHA family); MAC; digital signatures; certificates and PKI.	6
M5	**Protocols & Applications** TLS/SSL; authentication protocols; introduction to zero-knowledge proofs and blockchain; side-channel and attacks.	6
	Total Contact Hours (Lecture)	**32**

Text Books

1. W. Stallings, Cryptography and Network Security: Principles and Practice, Pearson.
2. J. Katz, Y. Lindell, Introduction to Modern Cryptography, CRC Press.

Reference Books

1. D. R. Stinson, M. Paterson, Cryptography: Theory and Practice, CRC Press.
2. B. Schneier, Applied Cryptography, Wiley.

Industry Relevance

Cryptography underpins secure communication, authentication, payments, and blockchain. These skills are essential for cyber-security, secure-software, and fintech roles, and align with national priorities in digital trust.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Cryptography and Network Security — https://nptel.ac.in/courses/106105162
- Dan Boneh Cryptography I (open materials) — https://crypto.stanford.edu/~dabo/courses/
- Cryptopals challenges — https://cryptopals.com/
- OpenSSL documentation — https://www.openssl.org/docs/

Suggested Laboratory Exercises

1. Implement classical ciphers and cryptanalysis.
2. AES encryption/decryption exercise.
3. RSA key generation and encryption.
4. Diffie-Hellman key-exchange implementation.
5. Hashing and digital-signature verification.

Open-Source Simulators / Tools

Python (PyCryptodome) • OpenSSL • SageMath • Wireshark (TLS)

CS-L-15-C Deep Learning

Course Code	CS-L-15-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-0
Prerequisite	Applied Machine Learning, Linear Algebra, Optimization	**Laboratory**	Mandatory

Covers neural-network architectures and training, including CNNs, RNNs, and modern deep-learning methods with hands-on framework practice.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Explain neural-network fundamentals and training.
CO2	Build and train deep feed-forward networks.
CO3	Apply convolutional networks to vision tasks.
CO4	Apply recurrent and sequence models.
CO5	Apply modern architectures and transfer learning.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Neural Network Basics** Perceptron; multilayer networks; activation functions; backpropagation; gradient descent and variants.	6
M2	**Training Deep Networks** Loss functions; regularization (dropout, batch norm); initialization; optimization; hyperparameter tuning.	6
M3	**Convolutional Networks** Convolution and pooling; CNN architectures (LeNet, ResNet); image classification; object detection (overview).	7
M4	**Sequence Models** Recurrent networks; LSTM and GRU; sequence-to-sequence; word embeddings; applications in NLP.	6
M5	**Modern Architectures** Attention and transformers (introduction); autoencoders; generative models (GANs overview); transfer learning; responsible AI.	7
	Total Contact Hours (Lecture)	**32**

Text Books

1. I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press.
2. A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow, O'Reilly.

Reference Books

1. F. Chollet, Deep Learning with Python, Manning.
2. C. C. Aggarwal, Neural Networks and Deep Learning, Springer.

Industry Relevance

Deep learning powers computer vision, natural-language processing, speech, and generative AI. These skills are central to AI/ML-engineer and research roles and to the current wave of AI-driven products.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Deep Learning — https://nptel.ac.in/courses/106106184
- Stanford CS231n (CNNs for vision) — https://cs231n.github.io/
- PyTorch tutorials — https://pytorch.org/tutorials/
- fast.ai Practical Deep Learning — https://course.fast.ai/

Suggested Laboratory Exercises

1. Build a feed-forward network from scratch.
2. Train a CNN for image classification.
3. RNN/LSTM for sequence prediction.
4. Transfer learning with a pretrained model.
5. Mini-project on a real dataset.

Open-Source Simulators / Tools

Python (PyTorch, TensorFlow/Keras) • Google Colab • Jupyter • NumPy

CS-L-07-C Computer Vision

Course Code	CS-L-07-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Applied Machine Learning, Linear Algebra	**Laboratory**	Mandatory

Covers image formation, processing, feature extraction, and modern deep-learning approaches to visual recognition and scene understanding.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Explain image formation and basic image processing.
CO2	Apply feature detection and description techniques.
CO3	Apply geometric vision and camera models.
CO4	Apply machine/deep learning to vision tasks.
CO5	Build computer-vision applications.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Image Formation & Processing** Image formation; cameras; sampling and quantization; filtering; edges; histograms; morphological operations.	6
M2	**Features & Matching** Corner and blob detection; SIFT/SURF/ORB; feature matching; image stitching; optical flow.	7
M3	**Geometric Vision** Camera models and calibration; epipolar geometry; stereo; structure from motion (introduction).	6
M4	**Recognition & Segmentation** Image classification; object detection (R-CNN, YOLO overview); semantic and instance segmentation; CNNs for vision.	7
M5	**Advanced Topics** Transfer learning; vision transformers (introduction); generative vision models; applications and ethics.	6
	Total Contact Hours (Lecture)	**32**

Text Books

1. R. Szeliski, Computer Vision: Algorithms and Applications, Springer.
2. D. A. Forsyth, J. Ponce, Computer Vision: A Modern Approach, Pearson.

Reference Books

1. R. C. Gonzalez, R. E. Woods, Digital Image Processing, Pearson.
2. R. Hartley, A. Zisserman, Multiple View Geometry in Computer Vision, Cambridge University Press.

Industry Relevance

Computer vision powers autonomous driving, medical imaging, surveillance, AR/VR, and quality inspection. CV skills map to vision-engineer, ML-engineer, and research roles across automotive, healthcare, and consumer tech.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Deep Learning for Computer Vision — https://nptel.ac.in/courses/106106224
- Stanford CS231n (open) — https://cs231n.github.io/
- OpenCV documentation & tutorials — https://docs.opencv.org/
- PyTorch vision (torchvision) tutorials — https://pytorch.org/vision/stable/index.html

Suggested Laboratory Exercises

1. Image filtering and edge detection.
2. Feature detection and matching (ORB/SIFT).
3. Camera calibration and stereo.
4. CNN image classification.
5. Object detection with a pretrained model.

Open-Source Simulators / Tools

Python (OpenCV, PyTorch, torchvision) • Google Colab • Jupyter

CS-L-17-C Contemporary Practices in Software Engineering

Course Code	CS-L-17-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Object Oriented Programming, Data Structures	**Laboratory**	Mandatory

Covers modern software-engineering processes, agile practices, version control, testing, CI/CD, and collaborative team-based development.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Apply software-development life-cycle and agile processes.
CO2	Elicit, analyse, and specify software requirements.
CO3	Apply software design, architecture, and patterns.
CO4	Apply testing, version control, and CI/CD practices.
CO5	Work effectively in a collaborative software team.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Processes & Agile** SDLC models; agile and Scrum; Kanban; DevOps culture; project planning and estimation.	6
M2	**Requirements & Modelling** Requirements engineering; user stories; UML modelling; use cases; specification and prioritization.	6
M3	**Design & Architecture** Software architecture; design patterns; SOLID principles; microservices vs monolith; API design.	7
M4	**Quality & Testing** Testing levels and types; unit/integration/system testing; test-driven development; code review; static analysis.	7
M5	**Version Control & CI/CD** Git workflows; branching strategies; continuous integration and delivery; containerized deployment; observability.	6
	Total Contact Hours (Lecture)	**32**

Text Books

1. I. Sommerville, Software Engineering, Pearson.
2. R. S. Pressman, B. R. Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill.

Reference Books

1. R. C. Martin, Clean Code, Prentice Hall.
2. M. Fowler, Refactoring: Improving the Design of Existing Code, Addison-Wesley.

Industry Relevance

Modern software-engineering practice — agile, Git, testing, and CI/CD — is the day-to-day reality of professional development teams. These skills are essential and immediately applicable in any software-engineering role.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Software Engineering — https://nptel.ac.in/courses/106105182
- Pro Git (free book) — https://git-scm.com/book/en/v2
- GitHub Actions documentation — https://docs.github.com/en/actions
- The Agile Manifesto & resources — https://agilemanifesto.org/

Suggested Laboratory Exercises

1. Git branching and collaborative workflow.
2. Requirements and UML modelling.
3. Unit testing and test-driven development.
4. CI/CD pipeline setup.
5. Team mini-project with agile sprints.

Open-Source Simulators / Tools

Git / GitHub • JUnit / PyTest • GitHub Actions • Docker • Jira / Trello

CS-L-19-C Distributed Systems

Course Code	CS-L-19-C	**Credits**	3
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	2-0-2-2
Prerequisite	Operating Systems, Computer Networks	**Laboratory**	Mandatory

Covers the principles of distributed systems: communication, coordination, consistency, replication, fault tolerance, and consensus.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Explain distributed-system models and challenges.
CO2	Apply inter-process communication and RPC.
CO3	Apply clock synchronization and coordination algorithms.
CO4	Explain replication, consistency, and fault tolerance.
CO5	Explain consensus and distributed-data techniques.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Foundations** Distributed-system models; architectures; challenges; communication paradigms; middleware.	6
M2	**Communication & Coordination** Remote procedure calls; message passing; logical and vector clocks; mutual exclusion; election algorithms.	7
M3	**Consistency & Replication** Replication models; consistency models; quorum systems; the CAP theorem; eventual consistency.	7
M4	**Fault Tolerance & Consensus** Failure models; reliable communication; consensus (Paxos, Raft overview); Byzantine fault tolerance; distributed transactions.	6
M5	**Distributed Data & Systems** Distributed file systems; MapReduce; distributed databases; introduction to blockchain as a distributed system.	6
	Total Contact Hours (Lecture)	**32**

Text Books

1. A. S. Tanenbaum, M. van Steen, Distributed Systems: Principles and Paradigms, Pearson.
2. G. Coulouris et al., Distributed Systems: Concepts and Design, Pearson.

Reference Books

1. M. Kleppmann, Designing Data-Intensive Applications, O'Reilly.
2. M. van Steen, A. S. Tanenbaum, Distributed Systems (free 3rd ed.).

Industry Relevance

Distributed-systems concepts underpin every large-scale internet service. Understanding consistency, replication, and consensus is essential for backend, cloud, and infrastructure-engineering roles at scale.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Distributed Systems — https://nptel.ac.in/courses/106106168
- MIT 6.824 Distributed Systems (open) — https://pdos.csail.mit.edu/6.824/
- The Raft consensus visualization — https://raft.github.io/
- Apache Hadoop / Spark documentation — https://hadoop.apache.org/docs/stable/

Suggested Laboratory Exercises

1. RPC-based client-server application.
2. Logical-clock and mutual-exclusion simulation.
3. Replication and consistency exercise.
4. Implement a simplified Raft/leader election.
5. MapReduce job on a small dataset.

Open-Source Simulators / Tools

Python / Go • gRPC • Apache Hadoop/Spark • Docker • etcd

CS-L-20-C Compiler Design

Course Code	CS-L-20-C	**Credits**	2
Category	PCC (Programme Core)	**Structure (L-T-P-S)**	1-0-2-0
Prerequisite	Finite Automata and Formal Languages, Data Structures	**Laboratory**	Mandatory

Covers the phases of compilation: lexical analysis, parsing, semantic analysis, intermediate code, optimization, and code generation.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Explain the structure and phases of a compiler.
CO2	Design lexical analysers using regular expressions.
CO3	Apply parsing techniques for context-free grammars.
CO4	Apply syntax-directed translation and semantic analysis.
CO5	Apply intermediate-code generation and optimization.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Introduction & Lexical Analysis** Phases of a compiler; lexical analysis; tokens and patterns; finite automata for scanners; lexer generators (lex/flex).	5
M2	**Syntax Analysis** Context-free grammars; top-down parsing (recursive descent, LL(1)); bottom-up parsing (LR, SLR, LALR); parser generators (yacc).	6
M3	**Semantic Analysis** Syntax-directed definitions; attribute grammars; type checking; symbol tables; scope handling.	5
M4	**Intermediate Code & Runtime** Intermediate representations; three-address code; runtime environments; activation records; storage organization.	5
M5	**Optimization & Code Generation** Code optimization techniques; basic blocks and flow graphs; register allocation; target code generation.	5
	Total Contact Hours (Lecture)	**26**

Text Books

1. A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, Compilers: Principles, Techniques, and Tools, Pearson.
2. K. D. Cooper, L. Torczon, Engineering a Compiler, Morgan Kaufmann.

Reference Books

1. A. W. Appel, Modern Compiler Implementation in C/Java, Cambridge University Press.
2. D. Grune et al., Modern Compiler Design, Springer.

Industry Relevance

Compiler concepts apply to language tooling, interpreters, static analysers, and domain-specific languages. The skills support roles in programming-languages, developer-tools, and performance engineering.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Compiler Design — https://nptel.ac.in/courses/106105190
- Stanford Compilers (open course) — https://online.stanford.edu/courses/soe-ycscs1-compilers
- LLVM tutorial (Kaleidoscope) — https://llvm.org/docs/tutorial/
- Crafting Interpreters (free book) — https://craftinginterpreters.com/

Suggested Laboratory Exercises

1. Lexical analyser with flex.
2. Parser with yacc/bison.
3. Symbol-table and type-checking implementation.
4. Three-address-code generation.
5. A simple optimization pass.

Open-Source Simulators / Tools

flex / bison • LLVM • GCC • Python (PLY)

7. Professional Electives (Programme Electives)

The programme offers Professional/Programme Elective courses (PE-L-xx-E, 3 credits each) that allow students to build depth in one of three specialisation tracks aligned with the department's strengths and current industry demand. Students choose Programme Electives across Semesters 6 and 7 as per the scheme. A student may concentrate in a single track or combine courses across tracks. The lists below are indicative; the department may add or revise courses from time to time.

Track A — Data Science & Artificial Intelligence

Aligned with the institute's strengths in AI/ML and data science and the national IndiaAI mission.

Sl.	Course Code	Course Title	Credits
1	PE-L-01-E	Natural Language Processing	3
2	PE-L-02-E	Computer Vision	3
3	PE-L-03-E	Reinforcement Learning	3
4	PE-L-04-E	Big Data Analytics	3
5	PE-L-05-E	Data Mining and Warehousing	3
6	PE-L-06-E	Generative AI and Large Language Models	3
7	PE-L-07-E	Time-Series and Predictive Analytics	3
8	PE-L-08-E	Explainable and Responsible AI	3

Track B — Networking & Cyber-Security

Focuses on advanced networking, security, and distributed-trust technologies for a connected world.

Sl.	Course Code	Course Title	Credits
1	PE-L-09-E	Advanced Computer Networks	3
2	PE-L-10-E	Network and Cyber Security	3
3	PE-L-11-E	Wireless and Mobile Networks	3
4	PE-L-12-E	Software-Defined Networking	3
5	PE-L-13-E	Blockchain Technology and Applications	3
6	PE-L-14-E	Ethical Hacking and Penetration Testing	3
7	PE-L-15-E	Information and Network Forensics	3
8	PE-L-16-E	Internet of Things and Edge Networks	3

Track C — Embedded & Systems

Builds depth in systems software, high-performance and embedded computing, and hardware-software co-design.

Sl.	Course Code	Course Title	Credits
1	PE-L-17-E	Real-Time and Embedded Systems	3
2	PE-L-18-E	Advanced Operating Systems	3
3	PE-L-19-E	Parallel and High-Performance Computing	3
4	PE-L-20-E	GPU Computing and CUDA Programming	3

Sl.	Course Code	Course Title	Credits
5	PE-L-21-E	Cyber-Physical Systems	3
6	PE-L-22-E	Embedded Machine Learning (TinyML)	3
7	PE-L-23-E	System-on-Chip and Hardware Acceleration	3
8	PE-L-24-E	Quantum Computing (Introduction)	3

8. Open Electives (Suggested)

Open Electives (OE-L-xx-E, 3 credits each) are taken in Semesters 6 and 7 and allow CSE students to broaden their competencies beyond the core discipline. The following are suggested Open Electives grouped by domain, drawn from sister departments (ECE/DSAI), the management group, and humanities & social sciences. The actual basket offered each term is published by the institute.

Domain I — Electronics / Data Science & AI

Sl.	Course Code	Course Title	Credits
1	OE-L-01-E	Digital Signal Processing	3
2	OE-L-02-E	VLSI Design (Introduction)	3
3	OE-L-03-E	Communication Systems	3
4	OE-L-04-E	Data Visualization and Storytelling	3
5	OE-L-05-E	Business Intelligence and Analytics	3
6	OE-L-06-E	Introduction to Robotics and Drones	3
7	OE-L-07-E	Bioinformatics and Computational Biology	3
8	OE-L-08-E	Human-Computer Interaction	3

Domain II — Management

Sl.	Course Code	Course Title	Credits
1	OE-L-09-E	Project Management	3
2	OE-L-10-E	Financial Management	3
3	OE-L-11-E	Marketing Management	3
4	OE-L-12-E	Organizational Behaviour	3
5	OE-L-13-E	Operations and Supply-Chain Management	3
6	OE-L-14-E	Technology and Innovation Management	3

Domain III — Humanities & Social Sciences

Sl.	Course Code	Course Title	Credits
1	OE-L-15-E	Technical Writing and Communication	3
2	OE-L-16-E	Introduction to Psychology	3
3	OE-L-17-E	Principles of Economics	3
4	OE-L-18-E	Philosophy of Science and Technology	3
5	OE-L-19-E	Sociology of Technology	3
6	OE-L-20-E	Science, Technology and Society	3

Note: Multidisciplinary Minor (MDM) streams — Digital Forensics, Business Informatics, Digital Humanities, Electric Vehicle Technology, Quantum Technologies, and IEV — are administered separately under the institute's minor framework and are intentionally excluded from this syllabus book, as per scope.

HS-L-08-C Introduction to Finance

Course Code	HS-L-08-C	**Credits**	1
Category	HSMC (AEC)	**Structure (L-T-P-S)**	1-0-0-0
Prerequisite	NIL	**Laboratory**	No

Introduces students to the fundamental concepts of personal and corporate finance, building the financial literacy needed to make informed decisions about savings, investment, credit, and basic business finance.

Course Outcomes (COs)

On successful completion of this course, the student will be able to:

CO	Statement
CO1	Explain core financial concepts, including the time value of money and interest computation.
CO2	Compare savings, investment, and insurance instruments based on risk and return.
CO3	Apply budgeting and financial-planning principles to personal financial decisions.
CO4	Interpret basic corporate financial statements and key financial ratios.

Detailed Syllabus (5 Modules)

Mod.	Content	Hours
M1	**Foundations of Finance** — Overview of the financial system and institutions; time value of money; simple and compound interest; present value and future value; inflation and purchasing power.	5
M2	**Personal Finance & Investment** — Savings and budgeting; banking and digital payment systems; insurance basics; introduction to mutual funds and the stock market; risk-return trade-off; retirement and tax planning basics.	5
M3	**Corporate Finance Basics** — Reading financial statements (balance sheet, P&L, cash flow); key financial ratios; sources of business finance (equity, debt); basics of capital budgeting.	4
	Total Contact Hours (Lecture)	**14**

Text Books

1. Prasanna Chandra, *Fundamentals of Financial Management*, McGraw-Hill.
2. Z. Bodie, A. Kane, A. Marcus, *Investments*, McGraw-Hill.

Reference Books

1. M.Y. Khan, P.K. Jain, *Financial Management*, McGraw-Hill.
2. Benjamin Graham, *The Intelligent Investor*, HarperBusiness.

Industry Relevance

Financial literacy is essential for engineers managing project budgets, evaluating ESOPs/stock compensation, and navigating startup finance and personal wealth planning over a technical career.

Open Educational Resources (OERs) & Open Repositories

- NPTEL: Financial Management — https://nptel.ac.in/courses/110105031
- SEBI Investor Education — https://investor.sebi.gov.in
- Khan Academy: Personal Finance — https://www.khanacademy.org/college-careers-more/personal-finance

Suggested Learning Activities

- Prepare a personal monthly budget and savings plan.
- Compute present/future value for a sample loan or investment.
- Analyze the balance sheet and P&L of a listed company (case study).

NEP-2020 Based B.Tech - CSE- UG curriculum

(With effect from July 2025)

Total Credit : 160

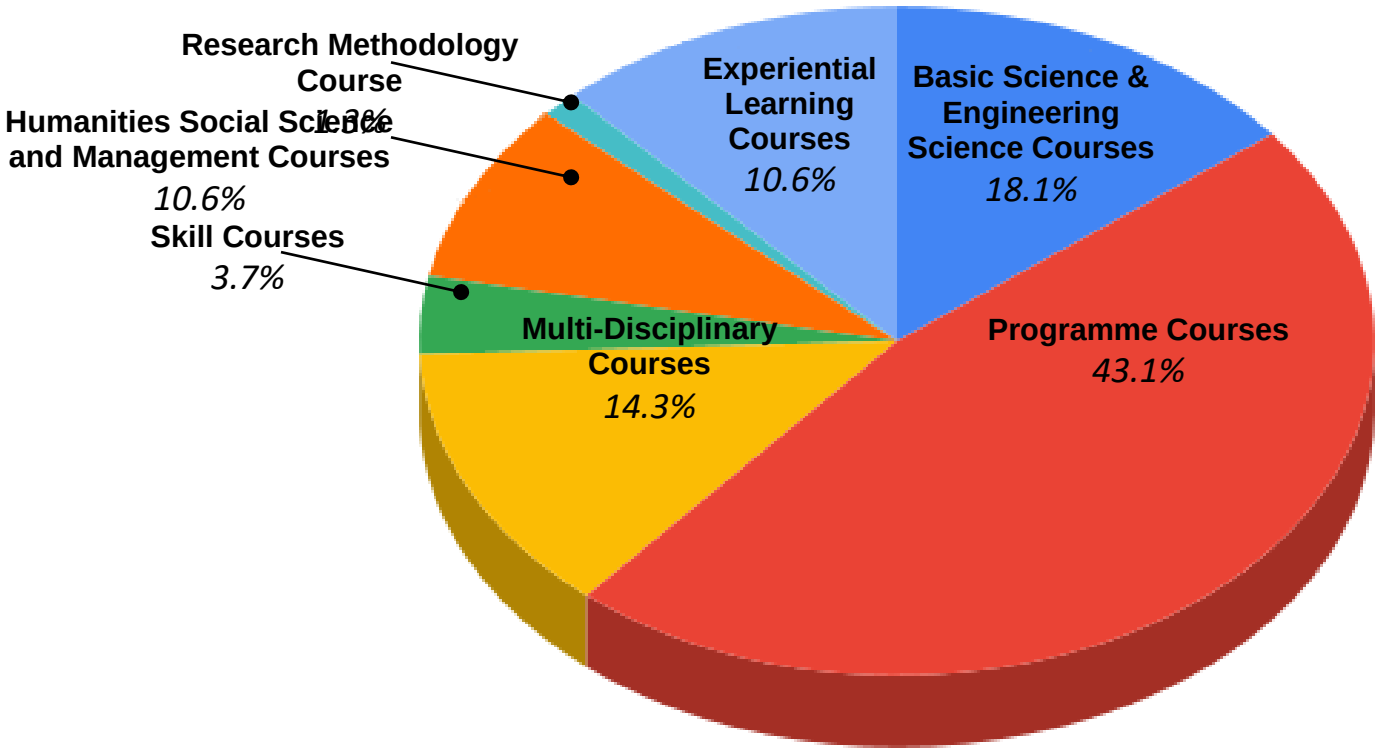
Lecture-L: 1 Credit = 1 hr/ week | Tutorial-T: 1 Credit = 1hr/week

Practical-P: 1 Credit = 2 hr/week | Scholarly Engagement Hours-S: 0 Credit= 2 hr/week

Multidisciplinary B.Tech - CSE Program: Sem-wise Credit Dist.- One Major, One Minor

Semester		I	II	III	IV	V	VI	VII	VIII	Total Credits	
Basic Science Course	BSC/ESC	06	06	03	--	--	--	--	--	**15**	**29**
Engineering Science Course		06	08		--	--	--	--	--	**14**	
Programme Core Course (PCC)	Programme Courses	--	--	15	19	15	05	–	--	**54**	**69**
Programme Elective Course (PEC)		--	--	--	--	–	09	06	--	**15**	
Open/School Elective (OE/SE) other than particular program	Multi-disciplinary	--	--	--	--	–	03	03	--	**06**	**18**
Multidisciplinary Minor (MD M)		--	--	03	03	03	03	03	--	**15**	
Vocational and Skill Enhancement Course (VSEC)	Skill Courses	02	04	--	–	--		--	--	**06**	**06**
Ability Enhancement Course (AEC-01)	Humanities Social Science and Management	02	02	--	--	--	--	--	--	**04**	**17**
Indian Language (AEC-02)		–	--	–	--	--	01	--	--	**01**	
Entrepreneurship/Economics/ Management Courses		02	--	--	–	02	--	–	--	**04**	
Indian Knowledge System (IKS)		–	–	02		--	--	02	--	**04**	
Value Education Course (VEC)		03	01	–	–	--	–	--	--	**04**	
Research Methodology (RM)	RM	--	--		--	02	--	–	--	**02**	**02**
Internship	Experiential Learning Courses	--	--		--	–	--	02			
Project		--	--		02	02	02	02	06	**16**	**17**

Community Engagement Activity (CEA)/Field Project		--	---	--	01	--	--	--	--	**01**	
Co-curricular & Extracurricular Activities (CCA)	Liberal Learning Course	–	–	--	--	--	02	--	--	**02**	**2**
Total Credits (One Major, One Minor degree)		**21**	**21**	**23**	**25**	**24**	**25**	**18**	**06**	**160**	**160**



Multidisciplinary Minor Streams

1. Cyber Security and Digital Forensics

Sl. No.	Name of the Course	Credits
1	Fundamentals of Cyber security	3
2	System Security	3
3	Cyber Threat Intelligence	3
4	Malware Analysis and Intrusion Detection Systems	3
5	Cyber Audit and Compliance	3

2. Business Informatics

Sl. No.	Name of the Course	Credits
1	Business Process Management	3
2	Introduction to FinTech	3
3	AI Based CRM	3
4	Digital Transformation using Process Mining	3
5	Strategic Supply Chain Management	3

3. Digital Humanities

Sl. No.	Name of the Course	Credits
1	Introduction to Digital Humanities	3
2	Digitalization of Heritage & Cultural Data	3
3	Digital Storytelling & Visualization	3
4	Culture Informatics: AI & Society	3
5	Spatial Humanities & GIS for Digital Mapping	3

4. Digital Mobility: Robotics and Unmanned Autonomous Systems

Sl. No.	Name of the Course	Credits
1	Fundamentals of Unmanned Autonomous System(UAS)	3
2	UAS Systems and Drone Technology	3
3	Autonomous Navigation and Motion Planning	3
4	UAS Data Analytics and Aerial Mapping	3
5	AI for Autonomous Systems	3

NEP Compliant Curriculum: HSM Courses to be included in UG Programs

Semester	Course category	Course Title	L-T-P-S	Credits	Course sub-category (*)
I	HSMC-01	International Language Competency	1-0-2-0	02	AEC
I	HSMC-02	Entrepreneurship and Design Thinking	1-0-0-0	01	-
I	HSMC-03	Human Values and Professional Ethics	2-0-0-0	02	VEC
I	HSMC-04	Constitution of India	1-0-0-0	01	VEC
II	HSMC-05	Communication & Discourse Strategies	2-0-0-0	02	AEC
II	HSMC-06	Environmental Studies	0-0-0-2	02	VEC
III	HSMC-07	Course on IKS in collaboration with Liberal Arts University	2-0-0-0	02	IKS
V	HSMC-08	Introduction to Finance	1-0-0-0	01	AEC
VI	HSMC-09	Indian/Foreign Language (One of the Following) Regional Language/German/Japanese	2-0-0-0	02	AEC
VII	HSMC-10	(One of the following) a) *History of Indian Civilizations* b) *Kautilya's Arthashastra* c) *Vedic Mathematics* d) *Vedic Corpus* e) *Wisdom from the Ages* f) *Panini's Grammar*	2-0-0-0	02	IKS
IV / VI	HSMC-11 / 12	Community Engagement Activities (NSS/ISR) / Extra and Co-curricular Activities	1 2	03	CEA CCA
		Total:	**13-0-2-2**	**20**	

- **AEC: Ability Enhancement Course**
- **IKS: Indian Knowledge System**
- **VEC: Value Education Course**
- **CEA: Community Engagement Activity**

Rubric Frameworks for Evaluation of Community Engagement Activities (NSS/ISR) / Extra and Co-Curricular Activities

Semester V

A. NSS (Hours of Service): NSS Coordinator will be responsible for providing the grade

Grade	A+	A	B+	B	C+	C	F
Hours of service	≥ 240	**200-239**	**180-199**	**140-179**	**100-139**	**60-99**	**<60**

B. ISR (30 Activities in Two years): ISR Coordinator will be responsible for providing the grade

Grade	A+	A	B+	B	C+	C	F
Present in % of activities	≥ 90%	**80%-89%**	**70%-79%**	**60%-69%**	**50%-59%**	**40%-49%**	**<40%**

a. Students can attend any one activity to earn the grade.
b. If a student wants to increase their grade, they need to participate in another activity also. Then the final grade will be decided by combining the grades of the two activities (NSS and ISR). For example, If a student scores an A in NSS and a C in ISR, his final grade will be A+. In general, students will be given one grade above the maximum grade of two activities, provided both grades are C and above.
c. For an F grade, the student has to do all ISR activities in the next semester.

Semester VI

Co-curricular & Extra-curricular Activities

Students' participation in the institute activities up to IV sem

a. Different clubs: technical and non-technical clubs; the coordinator of clubs will provide the names of students with attendance in the number of sessions and achievements
b. Sports activities: participation in institute sports activities and sports competition: The SAC president will be responsible for providing the participant's name
c. Attend IEEE activities and institute talk: The IEEE Coordinator will be responsible for providing the name
d. Volunteers in different technical and non-technical events. The faculty coordinator of particular activities will be responsible for providing the participant's name.

All activities will have equal weightage.

Total score (%)=$25\%(\frac{Total\ attendance}{Total\ number\ of\ session}+\frac{no\ times\ \ participation}{no\ of\ sport\ events}+\frac{No\ of\ talk\ attended}{Total\ number\ of\ talk}+\frac{No\ of\ event\ voluntered}{Total\ number\ of\ event})$

Grade	A+	A	B+	B	C+	C	F
Total score (%)	≥ 90%	**80%-89%**	**70%-79%**	**60%-69%**	**50%-59%**	**40%-49%**	**<40%**